

Python for AI

A Practical Feature Guide, Every Example Runnable

The Python language features that matter most for machine learning and AI engineering, each with a short, self-contained example whose output was captured by actually running the code.

Built with Claude Code

How to read this book

This guide is organised around Python *language features* rather than around any single AI library, because the features are what transfer across PyTorch, TensorFlow, JAX, scikit-learn, Hugging Face and the LLM API clients you will actually use. For every feature you get a one- or two-line explanation of what it is, why it shows up constantly in AI code, and a small program you can run.

Every code block marked with an OUTPUT panel was executed while this PDF was being generated. The green panel is the real captured output of the grey code above it, so if it prints here, it runs on your machine too (Python 3.10+ recommended; the numeric chapters use NumPy and pandas).

The examples are deliberately tiny and dependency-light. Where a line shows how a real framework uses a feature (for example a PyTorch `nn.Module` or an async LLM call), it is clearly marked as illustrative and is not executed.

Tip for Kindle: use the built-in table of contents (the ≡ navigation menu) to jump between chapters. Every chapter and part is a bookmark. Rotating to landscape enlarges the code panels.

Contents

Part I: Python Core for AI

1. Numbers, Booleans and the Object Model	9
2. Strings and f-strings for Text Data	11
3. Lists, Tuples, Sets and Dicts	13
4. Comprehensions and Generator Expressions	16
5. Control Flow, Truthiness, Unpacking and the Walrus	19
6. Structural Pattern Matching (match / case)	22

Part II: Functions and Functional Programming

7. Functions: Arguments, Defaults and the Mutable-Default...	25
8. Lambdas, map, filter and sorted(key=)	29
9. functools: partial, lru_cache, reduce, wraps	32
10. itertools: Batching and Combining Iterables	35

Part III: Iteration, Generators and Data Pipelines

11. Iterables, Iterators and the Iterator Protocol	39
12. Generators and yield: Streaming Datasets and Mini-Ba...	42
13. Generator Expressions, yield from and Lazy Pipelines	46

Part IV: Objects and Abstractions

14. Classes, Dunder Methods and __call__: the nn.Module...	50
15. Inheritance, super(), Abstract Base Classes and Prope...	53
16. Dataclasses for Configs and Records	56
17. Enums and NamedTuples	59
18. Decorators: Timing, Caching and no_grad-style Wrapp...	61
19. Context Managers and the with Statement	65

Part V: Typing and Robustness

20. Type Hints and the typing Module	69
21. Errors, Exceptions and Custom Exception Types	72
22. Logging Instead of print for Training Runs	75

Part VI: Data, Files and Serialization

23. File I/O and pathlib	79
--------------------------	----

24. JSON and CSV: Configs and Datasets	83
25. pickle and Model / Artifact Serialization	86
Part VII: Concurrency and Performance	
26. Threads, Processes and concurrent.futures for Data L... ..	90
27. async / await for Concurrent LLM and API Calls	94
Part VIII: The Numeric AI Stack	
28. NumPy Arrays, dtypes and Vectorization	99
29. Broadcasting, Indexing and Linear Algebra: the Math... ..	103
30. Random, Statistics and Activation Functions from Scr... ..	107
31. pandas Series and DataFrame: Loading Datasets	111
32. pandas Cleaning, Grouping and Feature Engineering	116
33. Visualization and the Deep-Learning Frameworks	120

PART I

Python Core for AI

Numbers, Booleans and the Object Model

Every value in Python is an **object** with a type, and even integers have unbounded precision. In AI you constantly move between Python's `int/float` and the fixed-width numeric types used by tensors (`float32`, `int64`), so it helps to know what the pure-Python side actually guarantees.

Integers never overflow

Unlike C or the fixed-width dtypes inside NumPy, a Python `int` grows as large as memory allows. This is why counting tokens or computing large factorials in plain Python never silently wraps around.

ARBITRARY PRECISION INTEGERS

```
n = 2 ** 200
print(n)
print('bit length:', n.bit_length())
# floor division and modulo, useful for batching
samples, batch = 1003, 32
print('full batches:', samples // batch, 'remainder:', samples % batch)
```

OUTPUT

```
1606938044258990275541962092341162602522202993782792835301376
bit length: 201
full batches: 31 remainder: 11
```

Floats, and why comparisons need tolerance

Floating-point math is inexact. Loss values and probabilities almost never compare exactly, so you test *closeness* with `math.isclose` (or `numpy.allclose` for arrays) instead of `==`.

NEVER COMPARE FLOATS WITH `==`

```
import math
a = 0.1 + 0.2
print('a =', a)
print('a == 0.3 ?', a == 0.3)
print('isclose ?', math.isclose(a, 0.3, rel_tol=1e-9))
```

OUTPUT

```
a = 0.30000000000000004
a == 0.3 ? False
isclose ? True
```

Booleans are integers

`True` and `False` are `1` and `0`. Summing a list of boolean conditions is the idiomatic way to *count* how many are true, which is exactly how you compute accuracy from a list of correct/incorrect predictions.

COUNTING CORRECT PREDICTIONS

```
preds = [1, 0, 1, 1, 0, 1]
labels = [1, 0, 0, 1, 0, 1]
correct = sum(p == y for p, y in zip(preds, labels))
accuracy = correct / len(labels)
print(f'{correct} correct out of {len(labels)}')
print(f'accuracy = {accuracy:.2%}')
```

OUTPUT

```
5 correct out of 6
accuracy = 83.33%
```

Why it matters for AI: metrics like accuracy, precision and recall are all just sums of boolean conditions divided by a count. Master `sum(cond for ...)` and you can write most metrics from scratch without a library.

CHAPTER 2

Strings and f-strings for Text Data

Text is the raw material of NLP and of every LLM prompt. Python strings are immutable Unicode sequences, and f-strings are the fastest, most readable way to build prompts and format numbers in logs.

f-strings: interpolation and formatting

BUILDING A PROMPT AND FORMATTING METRICS

```
role, topic = 'assistant', 'linear algebra'
prompt = f'You are a helpful {role}. Explain {topic} simply.'
print(prompt)

loss, lr = 0.03718, 3e-4
# format spec: fixed decimals, scientific, padding
print(f'loss={loss:.4f} lr={lr:.1e}')
for step in [1, 50, 1000]:
    print(f'step {step:>5} | {step/1000:6.1%} done')
```

OUTPUT

```
You are a helpful assistant. Explain linear algebra simply.
loss=0.0372 lr=3.0e-04
step    1 |  0.1% done
step   50 |  5.0% done
step 1000 | 100.0% done
```

The `=` specifier is a debugging superpower: `f'{x=}'` prints both the expression and its value.

SELF-DOCUMENTING DEBUG OUTPUT

```
temperature = 0.7
top_p = 0.95
print(f'{temperature=}, {top_p=}')
```

OUTPUT

```
temperature=0.7, top_p=0.95
```

Common text-cleaning methods

NORMALISING TEXT BEFORE TOKENISING

```
raw = ' Hello, WORLD! \n'
clean = raw.strip().lower().replace(',',' ')
print(repr(clean))
tokens = clean.split()
print('tokens:', tokens)
print('joined:', '|'.join(tokens))
print('starts with hi?', clean.startswith('hi'))
```

OUTPUT

```
'hello world!'
tokens: ['hello', 'world!']
joined: hello|world!
starts with hi? False
```

Why it matters for AI: prompt templates, chat formatting and cleaning scraped text are all string work. f-strings keep prompt-building readable, and `str.strip/split/join/replace` cover most preprocessing before a real tokenizer runs.

CHAPTER 3

Lists, Tuples, Sets and Dicts

Four built-in containers carry almost all data in Python before it becomes a tensor. Knowing which to reach for is half of writing clean data code: **lists** for ordered, mutable sequences (a batch of samples); **tuples** for fixed, immutable records (an array's [shape](#)); **sets** for fast membership and de-duplication (a vocabulary); **dicts** for keyed lookup (token to id, config values).

Lists: the workhorse sequence

BUILDING AND SLICING A LIST OF SAMPLES

```
data = [5, 2, 9, 1, 7, 3]
data.append(8)
print('list :', data)
print('first 3 :', data[:3])      # slice
print('last 2  :', data[-2:])
print('every 2nd:', data[::2])
print('reversed :', data[::-1])
```

OUTPUT

```
list : [5, 2, 9, 1, 7, 3, 8]
first 3 : [5, 2, 9]
last 2  : [3, 8]
every 2nd: [5, 9, 7, 8]
reversed : [8, 3, 7, 1, 9, 2, 5]
```

Tuples: immutable, hashable records

Tuples cannot change after creation, which makes them safe to use as dict keys and as the fixed shapes and coordinates you see all over NumPy and PyTorch.

TUPLES AS SHAPES AND DICT KEYS

```

shape = (32, 3, 224, 224) # batch, channels, H, W
batch, channels, h, w = shape # unpacking
print('channels =', channels, '| pixels =', h * w)
cache = {(0, 0): 'origin', (1, 2): 'point'} # tuple keys
print(cache[(1, 2)])

```

OUTPUT

```

channels = 3 | pixels = 50176
point

```

Sets: membership and de-duplication

BUILDING A VOCABULARY WITH A SET

```

words = ['cat', 'dog', 'cat', 'bird', 'dog', 'cat']
vocab = set(words)
print('unique   :', sorted(vocab))
print('size     :', len(vocab))
print("'cat' in?  :", 'cat' in vocab) # 0(1) lookup
a, b = {1, 2, 3}, {2, 3, 4}
print('overlap  :', a & b, '| union:', a | b)

```

OUTPUT

```

unique   : ['bird', 'cat', 'dog']
size     : 3
'cat' in? : True
overlap  : {2, 3} | union: {1, 2, 3, 4}

```

Dicts: keyed lookup and the config pattern

Dicts map keys to values with $O(1)$ lookup and preserve insertion order. They back token-to-id vocabularies, hyperparameter configs, and the **`**kwargs`** you pass into models.

A CONFIG DICT AND SAFE ACCESS

```

config = {'lr': 3e-4, 'epochs': 10, 'model': 'resnet'}
print('lr      : ', config['lr'])
print('dropout? : ', config.get('dropout', 0.0)) # default
config['batch_size'] = 64 # add a key
for key, value in config.items():
    print(f'  {key:12} = {value}')
stoi = {ch: i for i, ch in enumerate('abc')} # char -> id
print('token ids : ', stoi)

```

OUTPUT

```

lr      : 0.0003
dropout? : 0.0
  lr      = 0.0003
  epochs  = 10
  model   = resnet
  batch_size = 64
token ids : {'a': 0, 'b': 1, 'c': 2}

```

Why it matters for AI: a token-to-id **dict**, a **set** vocabulary, a **list** batch and a **tuple** shape appear in essentially every data-loading and tokenisation pipeline. Picking the right container makes that code both faster and clearer.

CHAPTER 4

Comprehensions and Generator Expressions

Comprehensions are Python's signature feature for building a collection from an iterable in one readable line. In AI code they replace most manual `for` loops used to transform, filter and reshape data, and they are faster than the equivalent loop with `.append()`.

List comprehensions: map and filter in one line

TRANSFORM AND FILTER TOGETHER

```
nums = [1, 2, 3, 4, 5, 6]
squares = [x * x for x in nums]
evens    = [x for x in nums if x % 2 == 0]
labels   = ['pos' if x > 3 else 'neg' for x in nums]
print('squares:', squares)
print('evens   :', evens)
print('labels  :', labels)
```

OUTPUT

```
squares: [1, 4, 9, 16, 25, 36]
evens   : [2, 4, 6]
labels  : ['neg', 'neg', 'neg', 'pos', 'pos', 'pos']
```

Nested comprehensions: flattening batches

FLATTEN A LIST OF SEQUENCES


```

batches = [[1, 2], [3, 4], [5, 6]]
flat = [x for batch in batches for x in batch]
print('flat  :', flat)
# a 3x3 identity-like grid
grid = [[1 if i == j else 0 for j in range(3)] for i in range(3)
        ]
for row in grid:
    print(row)

```

OUTPUT

```

flat  : [1, 2, 3, 4, 5, 6]
[1, 0, 0]
[0, 1, 0]
[0, 0, 1]

```

Dict and set comprehensions

INVERT A VOCABULARY; COLLECT LENGTHS

```

stoi = {'a': 0, 'b': 1, 'c': 2}
itos = {i: ch for ch, i in stoi.items()} # invert mapping
print('id->char:', itos)
words = ['hi', 'bye', 'hi', 'ok']
lengths = {w: len(w) for w in set(words)} # dict comp
print('lengths :', lengths)

```

OUTPUT

```

id->char: {0: 'a', 1: 'b', 2: 'c'}
lengths : {'hi': 2, 'ok': 2, 'bye': 3}

```

Generator expressions: lazy and memory-cheap

Swap the square brackets for parentheses and you get a **generator expression**: it produces items one at a time instead of building the whole list, so it uses almost no memory. This is how you stream over a dataset too large to fit in RAM, and it plugs straight into [sum](#), [max](#) and [any](#).

STREAMING AGGREGATION WITHOUT A LIST

```
import sys
n = 1_000_000
gen = (x * x for x in range(n))      # nothing computed yet
total = sum(gen)                     # consumed lazily
print('sum of squares:', total)
lst = [x * x for x in range(n)]
print('list bytes ~', sys.getsizeof(lst))
print('gen bytes ~', sys.getsizeof(x*x for x in range(n)))
```

OUTPUT

```
sum of squares: 333332833333500000
list bytes ~ 8448728
gen bytes ~ 208
```

Why it matters for AI: comprehensions are the idiomatic way to preprocess datasets (`[clean(x) for x in raw]`), and generator expressions let you stream huge corpora with a flat memory footprint. You will read and write these constantly.

CHAPTER 5

Control Flow, Truthiness, Unpacking and the Walrus

Beyond the obvious `if/for/while`, a handful of small Python conveniences show up so often in AI code that they are worth calling out on their own: truthiness, iterable unpacking, `enumerate/zip`, and the walrus operator.

Truthiness: empty things are False

Empty containers, `0`, and `None` are all *falsey*. This lets you write `if batch:` instead of `if len(batch) > 0:`. Note the important exception for arrays below.

FALSY VALUES IN GUARDS

```
for value in [[], [1], '', 'x', 0, 3, None]:
    print(f'{str(value)!r:>6} -> {bool(value)}')
batch = []
if not batch:
    print('skip: empty batch')
```

OUTPUT

```
'[]' -> False
'[1]' -> True
'' -> False
'x' -> True
'0' -> False
'3' -> True
'None' -> False
skip: empty batch
```

enumerate and zip: the loop duo

INDEXING AND PAIRING DURING ITERATION

```
names = ['loss', 'acc', 'lr']
vals  = [0.31, 0.88, 3e-4]
for i, (name, val) in enumerate(zip(names, vals)):
    print(f'{i}: {name:5} = {val}')
```

OUTPUT

```
0: loss  = 0.31
1: acc   = 0.88
2: lr    = 0.0003
```

Unpacking with * (star)

SPLITTING HEAD/TAIL AND MERGING

```
first, *rest = [10, 20, 30, 40]
print('first:', first, '| rest:', rest)
*init, last = [10, 20, 30, 40]
print('init :', init, '| last:', last)
a = [1, 2]; b = [3, 4]
merged = [*a, *b] # splat into a new list
cfg = {'**{'lr': 1e-3}, 'lr': 3e-4} # later key wins
print('merged:', merged, '| cfg:', cfg)
```

OUTPUT

```
first: 10 | rest: [20, 30, 40]
init : [10, 20, 30] | last: 40
merged: [1, 2, 3, 4] | cfg: {'lr': 0.0003}
```

The walrus operator := (assign inside an expression)

The walrus operator assigns and returns a value in the same expression, which avoids recomputing or duplicating a call, a common pattern in read-loops and early-stopping checks.

ASSIGN-AND-TEST IN ONE STEP

```
losses = [0.9, 0.6, 0.61, 0.60, 0.60]
best = float('inf')
for epoch, loss in enumerate(losses):
    if (improvement := best - loss) > 0.02:
        best = loss
        print(f'epoch {epoch}: improved by {improvement:.2f}')
    else:
        print(f'epoch {epoch}: no significant gain')
```

OUTPUT

```
epoch 0: improved by inf
epoch 1: improved by 0.30
epoch 2: no significant gain
epoch 3: no significant gain
epoch 4: no significant gain
```

Why it matters for AI: `enumerate/zip` drive training loops, `*`-unpacking merges configs and splits sequences, and the walrus keeps early-stopping and streaming reads concise. Caution: never use `if array:` on a NumPy/torch tensor with more than one element, it raises an ambiguity error, use `.any()/all()` instead.

CHAPTER 6

Structural Pattern Matching (match / case)

Introduced in Python 3.10, `match/case` deconstructs data by *shape*, not just by value. It is a clean way to dispatch on the many differently-shaped messages that flow through AI systems: chat turns, tool-call payloads, config variants and streaming API events.

Matching values and structures

DISPATCH ON A CHAT MESSAGE SHAPE

```
def render(msg):
    match msg:
        case {'role': 'system', 'content': c}:
            return f'[system] {c}'
        case {'role': 'user', 'content': c}:
            return f'user> {c}'
        case {'role': 'tool', 'name': n}:
            return f'(tool call: {n})'
        case _:
            return '<unknown>'
for m in [{'role': 'system', 'content': 'Be brief.'},
          {'role': 'user', 'content': 'Hi'},
          {'role': 'tool', 'name': 'search'}]:
    print(render(m))
```

OUTPUT

```
[system] Be brief.
user> Hi
(tool call: search)
```

Capturing, guards and sequence patterns

MATCH ON STRUCTURE WITH AN IF-GUARD

```
def describe(point):
    match point:
        case (0, 0):
            return 'origin'
        case (x, 0):
            return f'on x-axis at {x}'
        case (x, y) if x == y:
            return f'diagonal at {x}'
        case (x, y):
            return f'point ({x}, {y})'
for p in [(0, 0), (5, 0), (3, 3), (1, 2)]:
    print(p, '->', describe(p))
```

OUTPUT

```
(0, 0) -> origin
(5, 0) -> on x-axis at 5
(3, 3) -> diagonal at 3
(1, 2) -> point (1, 2)
```

Why it matters for AI: LLM tool-calling and streaming responses arrive as tagged, differently-shaped dicts/events. `match` reads far better than a long `if/elif` chain when you branch on the *structure* of a payload.

PART II

Functions and Functional Programming

CHAPTER 7

Functions: Arguments, Defaults and the Mutable-Default Trap

Functions are the unit of reuse in every AI codebase, from a one-line loss to a full training step. Python's calling conventions are flexible: positional and keyword arguments, defaults, `*args`, `**kwargs`, and keyword-only parameters. Knowing them well is what lets you pass a whole config into a model with `model(**config)`.

Positional, keyword and default arguments

Arguments can be given by position or by name. Defaults make parameters optional, which is how hyperparameters get sensible fallbacks.

DEFAULTS AND KEYWORD CALLS

```
def train_step(lr=1e-3, momentum=0.9, clip=None):
    msg = f"lr={lr}, momentum={momentum}, clip={clip}"
    return msg

print(train_step())           # all defaults
print(train_step(0.01))      # positional lr
print(train_step(clip=1.0, lr=0.1)) # keyword, any order
```

OUTPUT

```
lr=0.001, momentum=0.9, clip=None
lr=0.01, momentum=0.9, clip=None
lr=0.1, momentum=0.9, clip=1.0
```

*args and **kwargs

`*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dict. Frameworks use

them everywhere to forward options down a stack of layers without listing every parameter.

COLLECTING VARIADIC ARGUMENTS

```
def build_layer(name, *shape, **options):
    print("name :", name)
    print("shape :", shape)      # a tuple
    print("opts  :", options)    # a dict

    build_layer("dense", 784, 128, activation="relu", bias=False)
```

OUTPUT

```
name  : dense
shape : (784, 128)
opts  : {'activation': 'relu', 'bias': False}
```

Keyword-only arguments

Anything after a bare `*` in the signature must be passed by name. This prevents accidentally swapping two positional flags and makes call sites self-documenting.

FORCING CALLERS TO BE EXPLICIT

```
def sample(logits, *, temperature=1.0, greedy=False):
    return f"temp={temperature}, greedy={greedy}"

print(sample([0.1, 0.9], temperature=0.7))
try:
    sample([0.1, 0.9], 0.7)          # positional -> error
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
temp=0.7, greedy=False
TypeError: sample() takes 1 positional argument but 2 were given
```

The mutable-default trap

A default value is evaluated **once**, when the function is defined,

not on every call. A mutable default like `[]` is therefore *shared* across calls, a bug that silently corrupts accumulated data. The fix is the `None`-sentinel pattern.

THE BUG AND THE FIX

```
def buggy(x, acc=[]):          # DON'T: shared list
    acc.append(x)
    return acc

print("buggy:", buggy(1), buggy(2), buggy(3))

def fixed(x, acc=None):       # DO: fresh list each call
    if acc is None:
        acc = []
    acc.append(x)
    return acc

print("fixed:", fixed(1), fixed(2), fixed(3))
```

OUTPUT

```
buggy: [1, 2, 3] [1, 2, 3] [1, 2, 3]
fixed: [1] [2] [3]
```

Unpacking arguments at the call site

The mirror image of `*args/**kwargs`: use `*` to spread a sequence into positional arguments and `**` to spread a dict into keyword arguments. This is the idiom behind `model(**config)`.

SPREADING A CONFIG DICT

```
def make_optimizer(lr, weight_decay, betas):
    return f"Adam(lr={lr}, wd={weight_decay}, betas={betas})"

config = {"lr": 3e-4, "weight_decay": 0.01, "betas": (0.9, 0.999)}
print(make_optimizer(**config))    # dict -> keyword args

pair = (0.9, 0.999)
print("betas unpacked:", *pair)    # tuple -> positional args
```

OUTPUT

```
Adam(lr=0.0003, wd=0.01, betas=(0.9, 0.999))  
betas unpacked: 0.9 0.999
```

Why it matters for AI: config-driven training relies on ****config** unpacking, and the mutable-default trap is a real source of data-leak bugs (e.g. a metric list that never resets between runs). Prefer **None** sentinels for any list/dict/set default.

CHAPTER 8

Lambdas, map, filter and sorted(key=)

A `lambda` is a small anonymous function, most useful as the `key` = argument to `sorted`, `min` and `max`. Combined with `map` and `filter`, lambdas express data transforms compactly, though comprehensions are often clearer.

lambda basics

ANONYMOUS ONE-LINERS

```
square = lambda x: x * x
scale = lambda x, s=2.0: x * s
print(square(5), scale(3), scale(3, 10))
```

OUTPUT

```
25 6.0 30
```

map and filter (and why comprehensions win)

`map` applies a function to every item; `filter` keeps items where a predicate is true. Both return lazy iterators. For readability, a list/generator comprehension usually expresses the same thing more clearly.

MAP/FILTER VS COMPREHENSION

```
nums = [-2, -1, 0, 1, 2, 3]

# functional style
sq = list(map(lambda x: x * x, filter(lambda x: x > 0, nums)))
print("map/filter :", sq)

# comprehension style (usually preferred)
sq2 = [x * x for x in nums if x > 0]
print("comprehension:", sq2)
```

OUTPUT

```
map/filter : [1, 4, 9]
comprehension: [1, 4, 9]
```

sorted(key=): the workhorse

`key=` maps each element to a sort key. Sorting samples by sequence length (to build efficient batches) or ranking candidates by score are everyday uses.

SORT SAMPLES BY LENGTH

```
batch = ["hi", "hello there", "hey", "good morning all"]
by_len = sorted(batch, key=len)
print("shortest first:", by_len)
print("longest first :", sorted(batch, key=len, reverse=True))
```

OUTPUT

```
shortest first: ['hi', 'hey', 'hello there', 'good morning all']
longest first : ['good morning all', 'hello there', 'hey', 'hi']
```

Top-k with sorted, min and max

`max/min` also take `key=`. Selecting the best prediction or the top-k scoring candidates is a one-liner.

BEST AND TOP-K CANDIDATES

```
candidates = [("cat", 0.12), ("dog", 0.71), ("fox", 0.55), ("owl", 0.33)]

best = max(candidates, key=lambda c: c[1])
print("argmax:", best)

top2 = sorted(candidates, key=lambda c: c[1], reverse=True)[:2]
print("top-2 :", top2)
```

OUTPUT

```
argmax: ('dog', 0.71)
top-2 : [('dog', 0.71), ('fox', 0.55)]
```

Why it matters for AI: `sorted(..., key=...)` and `max(..., key=...)` implement argmax, ranking, top-k selection and length-bucketing without any library. Reach for a comprehension over `map/filter` unless you already have a named function to pass.

CHAPTER 9

functools: partial, lru_cache, reduce, wraps

The `functools` module contains higher-order tools that show up constantly: freezing arguments with `partial`, memoizing with `lru_cache`, folding sequences with `reduce`, and preserving metadata with `wraps`.

partial: freezing hyperparameters

`functools.partial` returns a new callable with some arguments already filled in. It is the clean way to preconfigure an optimizer, activation or metric before passing it somewhere that will call it with the remaining arguments.

PRE-BINDING ARGUMENTS

```
from functools import partial

def scaled_dropout(x, rate, training):
    return f"x={x}, rate={rate}, training={training}"

# freeze rate=0.1; callers only pass x and training
dropout = partial(scaled_dropout, rate=0.1)
print(dropout(1.0, training=True))
print(dropout(2.0, training=False))
```

OUTPUT

```
x=1.0, rate=0.1, training=True
x=2.0, rate=0.1, training=False
```

lru_cache / cache: memoization

`@lru_cache` stores results keyed by arguments, so repeated calls with the same inputs return instantly. Great for expensive pure functions like tokenizer lookups or recursive computations. `@cache` is an unbounded shortcut (Python 3.9+).

CACHE EXPENSIVE PURE CALLS

```

from functools import lru_cache

calls = {"n": 0}

@lru_cache(maxsize=None)
def fib(n):
    calls["n"] += 1
    return n if n < 2 else fib(n - 1) + fib(n - 2)

print("fib(30) =", fib(30))
print("actual calls:", calls["n"])    # linear, not exponential
print("cache info :", fib.cache_info())

```

OUTPUT

```

fib(30) = 832040
actual calls: 31
cache info : CacheInfo(hits=28, misses=31, maxsize=None, currsize=31)

```

reduce: folding a sequence

`functools.reduce` collapses a sequence to a single value by repeatedly applying a two-argument function. Useful for chaining transforms or multiplying a shape into a flat size.

PRODUCT OF A TENSOR SHAPE

```

from functools import reduce

shape = (32, 3, 224, 224)          # (batch, channels, H, W)
num_elements = reduce(lambda a, b: a * b, shape)
print("elements per batch tensor:", num_elements)

```

OUTPUT

```

elements per batch tensor: 4816896

```

wraps: honest decorators

When you write a decorator (see the Decorators chapter), the

wrapper replaces the original function and loses its name and docstring. `functools.wraps` copies that metadata back, which matters for debugging and introspection.

PRESERVING `__NAME__` AND `__DOC__`

```
from functools import wraps

def logged(fn):
    @wraps(fn)                    # copy metadata from fn
    def wrapper(*args, **kwargs):
        return fn(*args, **kwargs)
    return wrapper

@logged
def predict(x):
    "Run the model forward pass."
    return x

print("name:", predict.__name__)    # 'predict', not 'wrapper'
print("doc :", predict.__doc__)
```

OUTPUT

```
name: predict
doc : Run the model forward pass.
```

Why it matters for AI: `partial` preconfigures optimizers and transforms in data pipelines, `lru_cache` avoids recomputing tokenization or feature lookups, and `wraps` keeps your training-loop decorators debuggable.

CHAPTER 10

itertools: Batching and Combining Iterables

`itertools` provides memory-efficient building blocks for iteration. They operate lazily on streams, so you can process datasets far larger than RAM. The stars for AI work are `chain`, `islice`, `product`, `combinations` and `batched`.

chain: concatenate iterables

`chain` treats several iterables as one continuous stream, for example merging train shards without building a giant list.

MERGE DATASET SHARDS LAZILY

```
from itertools import chain

shard_a = ["a1", "a2"]
shard_b = ["b1", "b2", "b3"]
for i, item in enumerate(chain(shard_a, shard_b)):
    print(i, item)
```

OUTPUT

```
0 a1
1 a2
2 b1
3 b2
4 b3
```

islice and count: windows into infinite streams

`count` is an infinite counter and `islice` takes a finite slice from any iterator, so you can peek at the first N items of an endless generator without materialising it.

TAKE THE FIRST N OF A STREAM

```
from itertools import count, islice

steps = count(start=0, step=10)          # 0, 10, 20, ... forever
first5 = list(islice(steps, 5))
print("first 5 lr-warmup steps:", first5)
```

OUTPUT

```
first 5 lr-warmup steps: [0, 10, 20, 30, 40]
```

product: hyperparameter grids

`product` is the Cartesian product, i.e. nested loops without the nesting. It is the natural way to enumerate a grid search over hyperparameters.

GRID SEARCH OVER HYPERPARAMETERS

```
from itertools import product

lrs = [1e-2, 1e-3]
batch_sizes = [16, 32]
for lr, bs in product(lrs, batch_sizes):
    print(f"run: lr={lr}, batch_size={bs}")
```

OUTPUT

```
run: lr=0.01, batch_size=16
run: lr=0.01, batch_size=32
run: lr=0.001, batch_size=16
run: lr=0.001, batch_size=32
```

combinations: pairs without repetition

`combinations` yields unordered subsets, handy for building positive/negative pairs in contrastive learning or comparing every model against every other.

ALL UNIQUE MODEL PAIRS

```
from itertools import combinations

models = ["A", "B", "C"]
for m1, m2 in combinations(models, 2):
    print(f"compare {m1} vs {m2}")
```

OUTPUT

```
compare A vs B
compare A vs C
compare B vs C
```

batched: grouping a stream into mini-batches

Since Python 3.12, `itertools.batched` chops an iterable into tuples of a fixed size, which is exactly what a data loader does to turn a stream of samples into mini-batches. The final batch may be shorter.

STREAM -> MINI-BATCHES

```
from itertools import batched

samples = range(10)                                # pretend: 10 samples
for step, mb in enumerate(batched(samples, 4)):
    print(f"step {step}: batch={mb} (size {len(mb)})")
```

OUTPUT

```
step 0: batch=(0, 1, 2, 3) (size 4)
step 1: batch=(4, 5, 6, 7) (size 4)
step 2: batch=(8, 9) (size 2)
```

Why it matters for AI: `itertools` is the pure-Python toolkit for data pipelines. `chain` merges shards, `islice` bounds infinite generators, `product` enumerates hyperparameter grids, and `batched` builds mini-batches, all lazily and without loading everything into memory.

PART III

Iteration, Generators and Data Pipelines

CHAPTER 11

Iterables, Iterators and the Iterator Protocol

Every `for` loop in Python, including the one that walks your training data, is built on the *iterator protocol*. Understanding it lets you write custom data loaders that plug straight into `for batch in loader:` syntax.

`iter()` and `next()`

`iter(obj)` asks an *iterable* for an *iterator*; `next(it)` pulls the next value and raises `StopIteration` when it is exhausted. A `for` loop is just this in disguise.

DRIVING AN ITERATOR BY HAND

```
data = [10, 20, 30]
it = iter(data)
print(next(it))
print(next(it))
print(next(it))
try:
    next(it)
except StopIteration:
    print("StopIteration: stream exhausted")
```

OUTPUT

```
10
20
30
StopIteration: stream exhausted
```

How a `for`-loop really works

A `for` loop calls `iter()` once, then `next()` repeatedly, catching `StopIteration` to stop. This desugaring is worth seeing once so iterators feel less magical.

FOR-LOOP DESUGARED

```
def manual_for(iterable, body):
    it = iter(iterable)
    while True:
        try:
            item = next(it)
        except StopIteration:
            break
        body(item)

manual_for(["a", "b", "c"], lambda x: print("got", x))
```

OUTPUT

```
got a
got b
got c
```

`__iter__` and `__next__`: a custom iterator

An object is iterable if it defines `__iter__`; it is an iterator if it also defines `__next__`. Here is a mini data loader that yields shuffled batch *indices*, the same shape as a real [DataLoader](#).

A MINIMAL INDEX DATALOADER


```

import random

class BatchSampler:
    def __init__(self, n, batch_size, seed=0):
        self.idx = list(range(n))
        random.Random(seed).shuffle(self.idx)
        self.batch_size = batch_size

    def __iter__(self):
        self.pos = 0
        return self

    def __next__(self):
        if self.pos >= len(self.idx):
            raise StopIteration
        batch = self.idx[self.pos:self.pos + self.batch_size]
        self.pos += self.batch_size
        return batch

for batch in BatchSampler(7, batch_size=3):
    print("batch indices:", batch)

```

OUTPUT

```

batch indices: [4, 2, 1]
batch indices: [0, 5, 3]
batch indices: [6]

```

Why it matters for AI: PyTorch [Dataset/DataLoader](#), Hugging Face streaming datasets and tf.data all speak the iterator protocol.

Implement `__iter__`/`__next__` and your data source works with `for batch in loader:` and `enumerate` for free.

CHAPTER 12

Generators and yield: Streaming Datasets and Mini-Batches

A *generator* is the easiest way to build an iterator: write a normal function but use `yield` instead of `return`. Execution pauses at each `yield` and resumes on the next request, so values are produced lazily, one at a time, using almost no memory.

yield basics

Calling a generator function does not run it; it returns a generator object. Each `next()` runs until the following `yield`.

A GENERATOR COUNTS LAZILY

```
def warmup_lrs(base, steps):
    for s in range(1, steps + 1):
        yield base * s / steps          # produced on demand

gen = warmup_lrs(1e-3, 4)
print("type:", type(gen).__name__)
for lr in gen:
    print(f"lr = {lr:.4e}")
```

OUTPUT

```
type: generator
lr = 2.5000e-04
lr = 5.0000e-04
lr = 7.5000e-04
lr = 1.0000e-03
```

Laziness = memory efficiency

A generator never holds the whole sequence in memory. This lets you stream a dataset of millions of rows and process it one

record at a time, which a list of the same data could not fit.

STREAM A HUGE DATASET LAZILY

```
import sys

def read_huge_dataset(n):
    for i in range(n):                # pretend each line is read from disk
        yield f"sample_{i}"

stream = read_huge_dataset(10_000_000)
# only the generator object exists, not 10M strings
print("generator size (bytes):", sys.getsizeof(stream))
# consume just the first three without materialising the rest
for _ in range(3):
    print(next(stream))
```

OUTPUT

```
generator size (bytes): 208
sample_0
sample_1
sample_2
```

A mini-batch generator

Generators shine as batchers: pull items from an upstream iterable and `yield` fixed-size lists. Because it is lazy, it works on infinite or on-disk streams too.

GROUP A STREAM INTO BATCHES

```
def batches(iterable, size):
    batch = []
    for item in iterable:
        batch.append(item)
        if len(batch) == size:
            yield batch
            batch = []
    if batch:
        # yield the short final batch
        yield batch

for b in batches(range(7), 3):
    print("batch:", b)
```

OUTPUT

```
batch: [0, 1, 2]
batch: [3, 4, 5]
batch: [6]
```

Generators are single-use

A generator carries its own execution state and is exhausted after one full pass. To iterate again (a new epoch) you must create a fresh generator, a common source of silent empty second epochs.

EXHAUSTION AFTER ONE PASS

```
def gen():
    yield from range(3)

g = gen()
print("epoch 1:", list(g))
print("epoch 2:", list(g))          # empty: already exhausted
```

OUTPUT

```
epoch 1: [0, 1, 2]
epoch 2: []
```

Why it matters for AI: streaming data loaders, tokenizing corpora that do not fit in RAM, and yielding mini-batches are all generator patterns. Remember that a generator is single-use, so wrap it in a function you call once per epoch.

CHAPTER 13

Generator Expressions, `yield from` and Lazy Pipelines

Generator expressions give you a generator with comprehension syntax, and `yield from` delegates to a sub-generator. Together they let you assemble multi-stage data pipelines that stay lazy from end to end.

Generator expression vs list comprehension

Swap the `[]` of a list comprehension for `()` and you get a generator expression: same syntax, but it computes items on demand instead of building the whole list. For large data the memory difference is enormous.

`()` IS LAZY, `[]` IS EAGER

```
import sys

n = 1_000_000
listcomp = [x * x for x in range(n)]      # builds 1M ints now
genexpr = (x * x for x in range(n))      # builds nothing yet

print("list bytes:", sys.getsizeof(listcomp))
print("gen bytes:", sys.getsizeof(genexpr))
# feed a genexpr straight into a reducer, no intermediate list
print("sum of squares:", sum(x * x for x in range(n)))
```

OUTPUT

```
list bytes: 8448728
gen bytes: 208
sum of squares: 333332833333500000
```

Chaining generators into a pipeline

Because each stage consumes and produces a lazy stream, you can wire *read* -> *clean* -> *tokenize* -> *batch* together. Data flows one record at a time, so peak memory is just one record

plus one batch.

READ -> CLEAN -> TOKENIZE -> BATCH

```
def read(lines):
    for line in lines:
        yield line

def clean(stream):
    for line in stream:
        yield line.strip().lower()

def tokenize(stream):
    for line in stream:
        yield line.split()

def batch(stream, size):
    buf = []
    for item in stream:
        buf.append(item)
        if len(buf) == size:
            yield buf; buf = []
    if buf:
        yield buf

raw = [" Hello WORLD ", "Deep Learning", " GPUs go BRRR "]
pipeline = batch(tokenize(clean(read(raw))), size=2)
for b in pipeline:
    print(b)
```

OUTPUT

```
[['hello', 'world'], ['deep', 'learning']]
[['gpus', 'go', 'brrr']]
```

yield from: delegation and flattening

`yield from sub` yields every item of `sub` in turn, forwarding cleanly without a manual loop. It is the natural way to flatten nested shards into one stream.

FLATTEN SHARDS WITH YIELD FROM

```
def flatten(shards):
    for shard in shards:
        yield from shard          # splice each shard's items
        s in

shards = [["a", "b"], ["c"], ["d", "e", "f"]]
print("flattened:", list(flatten(shards)))
```

OUTPUT

```
flattened: ['a', 'b', 'c', 'd', 'e', 'f']
```

Why it matters for AI: lazy pipelines are how you preprocess corpora larger than memory. Use generator expressions to avoid materialising intermediates, compose stages as generators, and use `yield from` to flatten shards, keeping peak memory to a single record plus a batch.

PART IV

Objects and Abstractions

CHAPTER 14

Classes, Dunder Methods and `__call__`: the `nn.Module` Pattern

A **class** bundles data (attributes) with behaviour (methods).

Every neural-network layer you will ever use, `nn.Linear`, `nn.Conv2d`, a whole `nn.Module`, is just a class instance that holds parameters and knows how to transform an input.

`__init__` and instance vs class attributes

`__init__` runs when you build an instance. Attributes set on `self` are *per-instance*; attributes set on the class body are *shared* by every instance, handy for constants like a default learning rate.

INSTANCE VS CLASS ATTRIBUTES

```
class Neuron:
    activation = "relu"          # class attribute, shared

    def __init__(self, n_inputs):
        self.weights = [0.0] * n_inputs  # instance attribute

a, b = Neuron(3), Neuron(5)
print("a.weights:", a.weights)
print("b.weights:", b.weights)
print("shared activation:", a.activation, b.activation)
Neuron.activation = "gelu"      # change once, seen everywhere
print("after change:", a.activation, b.activation)
```

OUTPUT

```
a.weights: [0.0, 0.0, 0.0]
b.weights: [0.0, 0.0, 0.0, 0.0, 0.0]
shared activation: relu relu
after change: gelu gelu
```

`__repr__` makes objects printable

Dunder (double-underscore) methods hook into Python syntax. `__repr__` controls how an object shows up in the REPL and in logs, priceless when you print a model or a config.

A READABLE `__REPR__`

```
class Linear:
    def __init__(self, in_features, out_features):
        self.in_features = in_features
        self.out_features = out_features

    def __repr__(self):
        return f"Linear(in={self.in_features}, out={self.out_features})"

layer = Linear(768, 256)
print(layer)
print([Linear(4, 4), Linear(4, 1)])
```

OUTPUT

```
Linear(in=768, out=256)
[Linear(in=4, out=4), Linear(in=4, out=1)]
```

`__call__`: why `layer(x)` works

When an object defines `__call__`, you can invoke the instance like a function. This is exactly why `output = layer(x)` works in PyTorch instead of `layer.forward(x)`, the framework routes the call through `__call__`. Here is a pure-Python `Linear` layer that computes $y = xW + b$ and is *callable*.

A CALLABLE LINEAR LAYER (PURE PYTHON)

```

class Linear:
    def __init__(self, W, b):
        self.W = W          # out x in
        self.b = b          # length out

    def __call__(self, x):    # y = W @ x + b
        return [sum(w_i * x_j for w_i, x_j in zip(row, x)) + b_
                i
                for row, b_i in zip(self.W, self.b)]

layer = Linear(W=[[1.0, 0.0], [0.5, 0.5]], b=[0.1, -0.2])
x = [2.0, 4.0]
print("layer(x) =", layer(x))  # calls __call__, just like PyTorch

```

OUTPUT

```
layer(x) = [2.1, 2.8]
```

Why it matters for AI: the whole `model(x)` ergonomics of PyTorch and Keras rests on `__call__`. Understanding it demystifies why layers are 'called' and lets you write your own composable, callable components.

CHAPTER 15

Inheritance, `super()`, Abstract Base Classes and Properties

Inheritance lets a class reuse and specialise another. In practice you subclass a framework base (`nn.Module`, a scikit-learn `BaseEstimator`) and override one or two methods.

Subclassing and `super()`

`super().__init__(...)` runs the parent's setup so the child does not have to re-implement it, the same call you write at the top of every custom `nn.Module.__init__`.

SUPER() CALLS THE PARENT

```
class Layer:
    def __init__(self, name):
        self.name = name

    def forward(self, x):
        raise NotImplementedError

class ReLU(Layer):
    def __init__(self):
        super().__init__("relu")           # run parent setup
        self.calls = 0

    def forward(self, x):                   # override
        self.calls += 1
        return [max(0.0, v) for v in x]

r = ReLU()
print(r.name, "->", r.forward([-1.0, 2.0, -3.0]))
print("forward calls:", r.calls)
```

OUTPUT

```
relu -> [0.0, 2.0, 0.0]
forward calls: 1
```

Abstract base classes define a contract

`abc.ABC` plus `@abstractmethod` forces every subclass to implement required methods. Instantiating an incomplete subclass fails loudly, which is how frameworks guarantee your model actually has a `forward`.

@ABSTRACTMETHOD ENFORCES THE INTERFACE

```
from abc import ABC, abstractmethod

class Model(ABC):
    @abstractmethod
    def forward(self, x): ...

class Identity(Model):
    def forward(self, x):
        return x

print("Identity works:", Identity().forward(42))
try:
    Model()                                # abstract, cannot instantiate
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
Identity works: 42
TypeError: Can't instantiate abstract class Model without an im
    plementation for abstract method 'forward'
```

@property for computed attributes

A `@property` looks like an attribute but runs code on access, perfect for derived values such as a model's parameter count, which should never fall out of sync with the weights.

NUM_PARAMETERS AS A PROPERTY

```

class Linear:
    def __init__(self, in_f, out_f):
        self.in_f, self.out_f = in_f, out_f

    @property
    def num_parameters(self):
        return self.in_f * self.out_f + self.out_f  # weights
            + bias

layer = Linear(768, 256)
print("params:", layer.num_parameters)  # accessed like an attribute

```

OUTPUT

```
params: 196864
```

Why it matters for AI: every custom model you write subclasses a base and calls `super().__init__()`. ABCs catch missing methods before training starts, and properties keep derived stats (param counts, output shapes) always correct.

CHAPTER 16

Dataclasses for Configs and Records

Training runs are drowning in configuration: learning rate, batch size, number of layers. A `@dataclass` turns a plain class into a typed record with a free `__init__`, `__repr__` and equality, ideal for hyper-parameter configs.

A typed config with defaults

@DATACLASS WITH DEFAULTS

```
from dataclasses import dataclass

@dataclass
class TrainConfig:
    lr: float = 3e-4
    batch_size: int = 32
    epochs: int = 10

cfg = TrainConfig(lr=1e-3)
print(cfg)                    # auto __repr__
print("batch_size:", cfg.batch_size)
print("equal?", cfg == TrainConfig(lr=1e-3))
```

OUTPUT

```
TrainConfig(lr=0.001, batch_size=32, epochs=10)
batch_size: 32
equal? True
```

Mutable defaults need `default_factory`

A list or dict default must be created per instance with `field(default_factory=...)`, otherwise every config would *share* the same list, a classic bug.

FIELD(DEFAULT_FACTORY=LIST)


```

from dataclasses import dataclass, field, asdict

@dataclass
class ModelConfig:
    hidden_sizes: list = field(default_factory=lambda: [256, 128])
    dropout: float = 0.1

cfg = ModelConfig()
cfg.hidden_sizes.append(64)
print(cfg)
print("as dict:", asdict(cfg))    # easy to serialise to JSON

```

OUTPUT

```

ModelConfig(hidden_sizes=[256, 128, 64], dropout=0.1)
as dict: {'hidden_sizes': [256, 128, 64], 'dropout': 0.1}

```

frozen=True and __post_init__ validation

frozen=True makes a config immutable (hashable, safe to share), and **__post_init__** runs validation right after construction so bad hyper-parameters fail immediately.

IMMUTABLE, SELF-VALIDATING CONFIG

```

from dataclasses import dataclass

@dataclass(frozen=True)
class Config:
    lr: float
    def __post_init__(self):
        if not 0 < self.lr < 1:
            raise ValueError(f"lr out of range: {self.lr}")

print(Config(lr=0.01))
try:
    Config(lr=5.0)
except ValueError as e:
    print("rejected:", e)
try:
    Config(lr=0.01).lr = 0.02    # frozen, cannot mutate
except Exception as e:
    print(type(e).__name__, "on assignment")

```

OUTPUT

```

Config(lr=0.01)
rejected: lr out of range: 5.0
FrozenInstanceError on assignment

```

Why it matters for AI: Hugging Face [TrainingArguments](#) and most research configs are dataclasses. They give you typed, self-documenting, serialisable hyper-parameters with almost no boilerplate, and validation catches typos before a GPU-hour is wasted.

CHAPTER 17

Enums and NamedTuples

Magic strings ('cuda', 'adam') and bare tuples are easy to mistype and hard to read. Enums and named tuples give those values names and structure.

Enum for fixed choices

An `enum.Enum` defines a closed set of options, a device or an optimizer, so a typo becomes an error instead of a silent wrong run.

ENUM.ENUM FOR DEVICE SELECTION

```
from enum import Enum

class Device(Enum):
    CPU = "cpu"
    CUDA = "cuda"
    MPS = "mps"

def to(device: Device):
    print("moving tensors to", device.value)

to(Device.CUDA)
print("members:", [d.name for d in Device])
print("lookup:", Device("cpu"))
```

OUTPUT

```
moving tensors to cuda
members: ['CPU', 'CUDA', 'MPS']
lookup: Device.CPU
```

NamedTuple for lightweight records

A `typing.NamedTuple` is an immutable record with named fields. It is perfect for a training `Batch`: you get `batch.inputs` instead of `batch[0]`, but it still unpacks and behaves like a tuple.

TYPING.NAMEDTUPLE AS A BATCH

```

from typing import NamedTuple

class Batch(NamedTuple):
    inputs: list
    labels: list
    weight: float = 1.0

b = Batch(inputs=[1, 2, 3], labels=[0, 1, 1])
print("named access:", b.inputs, b.labels, b.weight)
x, y, w = b                                # still unpacks like a tuple
print("unpacked:", x, y, w)
print("as dict:", b._asdict())

```

OUTPUT

```

named access: [1, 2, 3] [0, 1, 1] 1.0
unpacked: [1, 2, 3] [0, 1, 1] 1.0
as dict: {'inputs': [1, 2, 3], 'labels': [0, 1, 1], 'weight': 1.0}

```

Why it matters for AI: dataloaders, RL transitions and model outputs are often named tuples (PyTorch returns them from many ops). Enums prevent `'cuda'/'Cuda'` typos and make valid options discoverable in your editor.

CHAPTER 18

Decorators: Timing, Caching and no_grad-style Wrappers

A **decorator** wraps a function to add behaviour without touching its body. You have already seen `@dataclass` and `@property`; the AI ecosystem is full of them: `@torch.no_grad()`, `@tf.function`, `@functools.lru_cache`.

A timing decorator with `functools.wraps`

`functools.wraps` copies the original function's name and docstring onto the wrapper so tracebacks and `help()` stay meaningful.

@TIMEIT MEASURES A FUNCTION

```
import time, functools

def timeit(fn):
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        out = fn(*args, **kwargs)
        dt = time.perf_counter() - t0
        print(f"{fn.__name__} took {dt*1e3:.2f} ms")
        return out
    return wrapper

@timeit
def matmul_size(n):
    return sum(i * i for i in range(n))

print("result:", matmul_size(100_000))
print("name preserved:", matmul_size.__name__)
```

OUTPUT

```
matmul_size took 3.67 ms
result: 333328333350000
name preserved: matmul_size
```

Caching repeated work

`functools.lru_cache` memoises results, turning an expensive recomputation into a dictionary lookup, useful for tokeniser lookups or pure feature functions.

@LRU_CACHE MEMOISES

```
from functools import lru_cache

calls = {"n": 0}

@lru_cache(maxsize=None)
def fib(n):
    calls["n"] += 1
    return n if n < 2 else fib(n - 1) + fib(n - 2)

print("fib(30) =", fib(30))
print("unique calls:", calls["n"])    # linear, not exponential
print(fib.cache_info())
```

OUTPUT

```
fib(30) = 832040
unique calls: 31
CacheInfo(hits=28, misses=31, maxsize=None, currsize=31)
```

A decorator that takes arguments

Add a layer of nesting and a decorator can accept parameters, for example a `@retry(n)` that re-runs a flaky call, exactly what you want around an unreliable model-serving API.

@RETRY(N) WITH AN ARGUMENT

```

import functools

def retry(times):
    def deco(fn):
        @functools.wraps(fn)
        def wrapper(*a, **k):
            for attempt in range(1, times + 1):
                try:
                    return fn(*a, **k)
                except Exception as e:
                    print(f"attempt {attempt} failed: {e}")
                    raise RuntimeError("all retries exhausted")
            return wrapper
        return deco

state = {"n": 0}

@retry(3)
def flaky():
    state["n"] += 1
    if state["n"] < 3:
        raise ValueError("temporary")
    return "ok"

print("result:", flaky())

```

OUTPUT

```

attempt 1 failed: temporary
attempt 2 failed: temporary
result: ok

```

In real code you rarely write these, you *apply* them. The snippet below is **illustrative** (PyTorch is not installed): `@torch.no_grad()` wraps a function so it runs without building the autograd graph, saving memory during evaluation.

FRAMEWORK DECORATORS (ILLUSTRATIVE)

```
import torch

@torch.no_grad()                # disables gradient tracking for the call
def evaluate(model, loader):
    model.eval()
    return sum(accuracy(model(x), y) for x, y in loader)

@torch.compile                  # JIT-optimize the function/module
def forward(x):
    return model(x)
```

Why it matters for AI: decorators are the vocabulary of modern frameworks. Recognising `@no_grad`, `@lru_cache`, `@compile` and `@jit` lets you read library code, and writing your own `@timeit`/`@retry` cleans up training and serving scripts.

CHAPTER 19

Context Managers and the with Statement

A **context manager** guarantees setup and teardown around a block, even if it raises. Files are the classic case, but in AI the pattern is everywhere: `with torch.no_grad():`, autocast, distributed scopes, timers.

with for files closes them for you

WITH ALWAYS CLOSES THE FILE

```
import tempfile, os

path = os.path.join(tempfile.gettempdir(), "metrics.txt")
with open(path, "w") as f:
    f.write("epoch,loss\n0,0.91\n1,0.42\n")
# file is guaranteed closed here, even on error

with open(path) as f:
    print(f.read().strip())
os.remove(path)
```

OUTPUT

```
epoch,loss
0,0.91
1,0.42
```

Writing your own with __enter__/_exit__

Any class with `__enter__` and `__exit__` is a context manager. A block timer is a tidy example, `__exit__` runs whether or not the body succeeded.

A TIMER CONTEXT MANAGER

```
import time

class Timer:
    def __init__(self, label):
        self.label = label
    def __enter__(self):
        self.t0 = time.perf_counter()
        return self
    def __exit__(self, exc_type, exc, tb):
        dt = time.perf_counter() - self.t0
        print(f"{self.label}: {dt*1e3:.2f} ms")
        return False # do not suppress exceptions

with Timer("training step"):
    total = sum(i * i for i in range(200_000))
print("total:", total)
```

OUTPUT

```
training step: 8.49 ms
total: 2666646666700000
```

contextlib.contextmanager: the easy way

The `@contextmanager` decorator lets you write a context manager as a generator: code before `yield` is setup, code after is teardown. Here we temporarily set and restore a random seed, a common trick for reproducible sampling.

@CONTEXTMANAGER FOR A TEMPORARY SEED

```

import random
from contextlib import contextmanager

@contextmanager
def temporary_seed(seed):
    state = random.getstate()    # setup: save
    random.seed(seed)
    try:
        yield
    finally:
        random.setstate(state)    # teardown: restore

with temporary_seed(0):
    print("seeded:", [random.randint(0, 9) for _ in range(4)])
with temporary_seed(0):
    print("repeat:", [random.randint(0, 9) for _ in range(4)])

```

OUTPUT

```

seeded: [6, 6, 0, 4]
repeat: [6, 6, 0, 4]

```

The same protocol powers the most common inference idiom. This snippet is **illustrative** (PyTorch not installed): the `with` block disables gradient tracking just for its body, then restores it.

WITH TORCH.NO_GRAD() (ILLUSTRATIVE)

```

import torch

with torch.no_grad():           # __enter__ turns grad off
    logits = model(inputs)      # no autograd graph built -> ↩
    less memory
    preds = logits.argmax(dim=-1)
# __exit__ restores gradient tracking here

```

Why it matters for AI: `with torch.no_grad()`, `with autocast()` and `with device:` are all this one protocol. Knowing it means you can write reproducible-seed, timing and resource-cleanup blocks that never leak state.

PART V

Typing and Robustness

CHAPTER 20

Type Hints and the typing Module

Type hints annotate what a function expects and returns. Python does **not** enforce them at runtime, they are for humans, editors and tools like `mypy`, but modern ML libraries lean on them heavily for autocomplete and clarity.

Function and variable annotations

Since Python 3.9 you write built-in generics directly: `list[int]`, `dict[str, float]`, `tuple[int, int]`. They document shapes-of-data without any import.

ANNOTATIONS ARE DOCUMENTATION

```
def normalise(xs: list[float]) -> list[float]:
    total: float = sum(xs)
    return [x / total for x in xs]

probs = normalise([2.0, 3.0, 5.0])
print("probs:", probs, "sum:", sum(probs))
print("annotations:", normalise.__annotations__)
```

OUTPUT

```
probs: [0.2, 0.3, 0.5] sum: 1.0
annotations: {'xs': list[float], 'return': list[float]}
```

Optional, Union and the | syntax

`X | None` (equivalently `Optional[X]`) marks a value that may be missing, a device that defaults to auto-detect, say. `A | B` means 'either type'.

OPTIONAL PARAMETERS WITH | NONE

```
def build_model(hidden: int, device: str | None = None) -> dict:
    :
    if device is None:
        device = "cpu"          # auto-detect fallback
    return {"hidden": hidden, "device": device}

print(build_model(256))
print(build_model(256, device="cuda"))
```

OUTPUT

```
{'hidden': 256, 'device': 'cpu'}
{'hidden': 256, 'device': 'cuda'}
```

Callable and simple generics

Callable types a function argument (an activation or a metric passed in), and a **TypeVar** writes a function that works for *any* type while keeping the relationship between input and output.

CALLABLE AND TYPEVAR

```
from typing import Callable, TypeVar

def apply(fn: Callable[[float], float], xs: list[float]) -> list[
    float]:
    return [fn(x) for x in xs]

relu = lambda x: max(0.0, x)
print("relu:", apply(relu, [-1.0, 0.5, 2.0]))

T = TypeVar("T")
def first(seq: list[T]) -> T:
    return seq[0]

print("first int:", first([10, 20]))
print("first str:", first(["a", "b"]))
```

OUTPUT

```
relu: [0.0, 0.5, 2.0]
first int: 10
first str: a
```

Hints are not enforced at runtime

This surprises newcomers: passing the wrong type still runs. Use hints for tooling, and add explicit checks (next chapter) when you truly need runtime guarantees.

THE HINT IS A LIE THE RUNTIME IGNORES

```
from typing import get_type_hints

def scale(x: int) -> int:
    return x * 3

print("wrong type still runs:", scale("ab"))    # 'ababab'
print("declared hints:", get_type_hints(scale))
```

OUTPUT

```
wrong type still runs: ababab
declared hints: {'x': <class 'int'>, 'return': <class 'int'>}
```

Why it matters for AI: framework APIs (PyTorch, Hugging Face, Pydantic-based configs) are richly typed, so hints power your editor's autocomplete and catch shape/argument mistakes before you launch an expensive run. Combine them with dataclasses for self-documenting configs.

CHAPTER 21

Errors, Exceptions and Custom Exception Types

AI code fails in predictable ways: mismatched tensor shapes, missing files, out-of-memory. Handling exceptions well means your training loop can log, recover, or fail with a message that actually tells you what went wrong.

try / except / else / finally

`else` runs only if no exception fired; `finally` always runs, ideal for releasing a resource such as a GPU handle or a file.

THE FULL TRY STRUCTURE

```
def safe_divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("cannot divide by zero")
        return None
    else:
        print("division ok")
        return result
    finally:
        print("cleanup always runs")

print("=>", safe_divide(10, 2))
print("=>", safe_divide(10, 0))
```

OUTPUT

```
division ok
cleanup always runs
=> 5.0
cannot divide by zero
cleanup always runs
=> None
```


Catch specific exceptions, not everything

Catch the narrowest exception you can handle. A bare `except:` hides real bugs; naming the type keeps unexpected errors loud.

HANDLE ONLY WHAT YOU EXPECT

```
config = {"lr": 0.001}
for key in ["lr", "batch_size"]:
    try:
        print(f"{key} = {config[key]}")
    except KeyError:
        print(f"{key} missing, using default")
```

OUTPUT

```
lr = 0.001
batch_size missing, using default
```

Custom exceptions for domain errors

Defining your own exception type makes failures self-describing and lets callers catch exactly your error. A shape-mismatch check is the canonical example.

A SHAPEMISMATCHERROR

```
class ShapeMismatchError(ValueError):
    """Raised when two tensor shapes are incompatible."""

def matmul_shapes(a, b):
    if a[1] != b[0]:
        raise ShapeMismatchError(f"cannot matmul {a} with {b}")
    return (a[0], b[1])

print("ok:", matmul_shapes((4, 8), (8, 2)))
try:
    matmul_shapes((4, 8), (16, 2))
except ShapeMismatchError as e:
    print("caught:", e)
```

OUTPUT

```
ok: (4, 2)
caught: cannot matmul (4, 8) with (16, 2)
```

Exception chaining with raise ... from

`raise NewError(...) from original` preserves the underlying cause, so a friendly, high-level message keeps the low-level traceback for debugging.

WRAP A LOW-LEVEL ERROR CLEARLY

```
class DataLoadError(Exception):
    pass

def load_batch(raw):
    try:
        return int(raw)
    except ValueError as e:
        raise DataLoadError(f"bad sample: {raw!r}") from e

try:
    load_batch("not-a-number")
except DataLoadError as e:
    print("high level:", e)
    print("caused by :", type(e.__cause__).__name__)
```

OUTPUT

```
high level: bad sample: 'not-a-number'
caused by : ValueError
```

Why it matters for AI: the most common PyTorch error you will ever read is a shape mismatch. Validating shapes early with a clear custom exception, and chaining low-level errors into readable ones, turns cryptic stack traces into a two-second fix.

CHAPTER 22

Logging Instead of print for Training Runs

`print` is fine for a quick script, but real training loops need timestamps, severity levels and the ability to silence or redirect output. The `logging` module gives you all of that with one configuration call.

Levels and basicConfig

Messages have levels (`DEBUG` < `INFO` < `WARNING` < `ERROR`). You set a threshold once; anything below it is dropped, so you can dial verbosity up or down without deleting log lines.

CONFIGURE LOGGING ONCE

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(levelname)s | %(message)s",
)
log = logging.getLogger("train")

log.debug("this is hidden (below INFO)")
log.info("starting training")
log.warning("learning rate looks high")
```

OUTPUT

```
INFO | starting training
WARNING | learning rate looks high
```

Logging per-step metrics

Inside a loop, logging structured metrics beats printing: you get consistent formatting and can later route the same lines to a file or a service without touching the loop body.

A TINY TRAINING LOOP WITH LOGGING

```
import logging

logging.basicConfig(level=logging.INFO,
                    format="%(levelname)s | step=%(message)s")
log = logging.getLogger("loop")

loss = 1.0
for step in range(1, 4):
    loss *= 0.5                    # pretend the model improves
    log.info("%d loss=%.4f", step, loss)
log.info("done, final loss=%.4f", loss)
```

OUTPUT

```
INFO | step=1 loss=0.5000
INFO | step=2 loss=0.2500
INFO | step=3 loss=0.1250
INFO | step=done, final loss=0.1250
```

Timestamps and richer formats

Adding `asctime` and the logger name to the format is what makes logs from long runs actually searchable after the fact.

TIMESTAMPED, NAMED LOGS

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(name)s] %(levelname)s: %(message)s",
    datefmt="%H:%M:%S",
    force=True,                    # override any earlier config
)
logging.getLogger("data").info("loaded %d samples", 50000)
logging.getLogger("model").warning("checkpoint dir is nearly full")
```

OUTPUT

```
17:27:12 [data] INFO: loaded 50000 samples
17:27:12 [model] WARNING: checkpoint dir is nearly full
```

Why it matters for AI: training runs last hours and produce thousands of lines. Levels let you keep DEBUG detail available but quiet by default, and structured, timestamped logs are what you grep through when a run diverges at 3 a.m. Frameworks and experiment trackers all build on [logging](#).

PART VI

Data, Files and Serialization

CHAPTER 23

File I/O and pathlib

Training runs live and die by files: datasets on disk, checkpoints, logs, configs. Python's `open()` plus the modern `pathlib` module give you a clean, cross-platform way to read, write and discover them without fragile string surgery on paths.

Reading and writing with a context manager

Always open files inside a `with` block. The file is closed automatically even if an exception fires mid-write, so you never leak a half-flushed checkpoint. The mode string picks the operation: `'w'` write, `'r'` read, `'a'` append.

WRITE THEN READ A TEXT FILE

```
import tempfile, os

path = os.path.join(tempfile.mkdtemp(), "notes.txt")
with open(path, "w") as f:
    f.write("epoch 1 loss 0.42\n")
    f.write("epoch 2 loss 0.31\n")

with open(path) as f:
    content = f.read()
    print(content, end="")
os.remove(path)
```

OUTPUT

```
epoch 1 loss 0.42
epoch 2 loss 0.31
```

Iterating lines lazily

A file object is itself an iterator: looping over it yields one line at a time without loading the whole file into memory. This is how you stream a multi-gigabyte `.jsonl` dataset that would never fit in RAM.

STREAM A LARGE FILE LINE BY LINE

```
import tempfile, os

path = os.path.join(tempfile.mkdtemp(), "data.txt")
with open(path, "w") as f:
    for i in range(5):
        f.write(f"sample-{i}\n")

# Never loads the whole file at once - one line per iteration.
with open(path) as f:
    for lineno, line in enumerate(f, start=1):
        print(lineno, line.strip())
os.remove(path)
```

OUTPUT

```
1 sample-0
2 sample-1
3 sample-2
4 sample-3
5 sample-4
```

pathlib: paths as objects

`pathlib.Path` treats a path as an object with useful attributes and methods. The `/` operator joins path parts, `.stem/.suffix` split a filename, and `.mkdir(parents=True, exist_ok=True)` creates directory trees safely.

BUILDING AND INSPECTING PATHS


```

from pathlib import Path
import tempfile

root = Path(tempfile.mkdtemp())
ckpt_dir = root / "checkpoints"
ckpt_dir.mkdir(parents=True, exist_ok=True)

ckpt = ckpt_dir / "model_epoch3.pt"
ckpt.write_bytes(b"fake-weights")
print("name :", ckpt.name)
print("stem :", ckpt.stem)
print("suffix:", ckpt.suffix)
print("exists:", ckpt.exists())
print("parent:", ckpt.parent.name)

```

OUTPUT

```

name : model_epoch3.pt
stem : model_epoch3
suffix: .pt
exists: True
parent: checkpoints

```

Globbering for checkpoints

`Path.glob` finds files by pattern, perfect for locating every checkpoint or shard in a directory. Combine it with `sorted` and `max` to grab the latest one.

FIND ALL CHECKPOINTS, PICK THE NEWEST

```

from pathlib import Path
import tempfile

d = Path(tempfile.mkdtemp())
for epoch in (1, 5, 12, 3):
    (d / f"ckpt_{epoch:03d}.pt").write_bytes(b"x")

ckpts = sorted(d.glob("ckpt_*.pt"))
print("found:", [p.name for p in ckpts])
latest = max(ckpts, key=lambda p: int(p.stem.split("_")[1]))
print("latest:", latest.name)

```

OUTPUT

```
found: ['ckpt_001.pt', 'ckpt_003.pt', 'ckpt_005.pt', 'ckpt_012.
      pt']
latest: ckpt_012.pt
```

Why it matters for AI: dataset loaders stream files line by line to stay within memory, and resuming training means globbing a checkpoint directory and loading the newest file. [pathlib](#) makes both robust across Linux, macOS and Windows.

CHAPTER 24

JSON and CSV: Configs and Datasets

Two text formats dominate practical AI work: **JSON** for configs, API payloads and `.jsonl` datasets, and **CSV** for tabular data. Both ship in the standard library, so you can read and write them without any dependency.

JSON: strings to objects and back

`json.dumps` serialises a Python object to a string (use `indent=` for a readable config file); `json.loads` parses it back. The `dump/load` pair does the same directly to and from file objects.

SERIALISE AND PARSE A CONFIG

```
import json

config = {"model": "gpt", "layers": 12, "lr": 3e-4, "dropout": 0.1}

text = json.dumps(config, indent=2)
print(text)

restored = json.loads(text)
print("layers x2 =", restored["layers"] * 2)
```

OUTPUT

```
{
  "model": "gpt",
  "layers": 12,
  "lr": 0.0003,
  "dropout": 0.1
}
layers x2 = 24
```

JSON Lines: one example per line

The `.jsonl` format stores one JSON object per line. It is the

de-facto standard for LLM fine-tuning datasets because you can stream it line by line and append new records cheaply.

WRITE AND READ A .JSONL DATASET

```
import json, io

records = [
    {"prompt": "2+2?", "completion": "4"},
    {"prompt": "capital of France?", "completion": "Paris"},
]

buf = io.StringIO()
for r in records:
    buf.write(json.dumps(r) + "\n")

buf.seek(0)
for line in buf:
    obj = json.loads(line)
    print(obj["prompt"], "->", obj["completion"])
```

OUTPUT

```
2+2? -> 4
capital of France? -> Paris
```

CSV with DictReader / DictWriter

The `csv` module handles quoting and escaping for you. `DictWriter` writes rows from dictionaries and `DictReader` yields each row as a dict keyed by the header, which keeps column access readable.

ROUND-TRIP A SMALL DATASET

```
import csv, io

rows = [
    {"text": "great movie", "label": 1},
    {"text": "boring plot", "label": 0},
]

buf = io.StringIO()
writer = csv.DictWriter(buf, fieldnames=["text", "label"])
writer.writeheader()
writer.writerows(rows)

buf.seek(0)
reader = csv.DictReader(buf)
pos = sum(1 for row in reader if row["label"] == "1")
print("CSV written:")
print(buf.getvalue(), end="")
print("positive examples:", pos)
```

OUTPUT

```
CSV written:
text,label
great movie,1
boring plot,0
positive examples: 1
```

Why it matters for AI: fine-tuning datasets are almost always `.jsonl`, hyperparameter configs are JSON or YAML, and classic tabular ML lives in CSV. Note that `csv` reads every field as a **string** - cast to `int/float` yourself before training.

CHAPTER 25

pickle and Model / Artifact Serialization

JSON only handles simple types. To save an arbitrary Python object - a fitted scaler, a vocabulary, a dict of NumPy arrays - you reach for **pickle**, Python's native binary serialisation format. It is what most ML checkpoint formats are built on under the hood.

Pickling arbitrary objects

`pickle.dumps` turns almost any object into bytes and `pickle.loads` reconstructs it. Here we save a small preprocessing artifact: a vocabulary plus normalisation statistics.

ROUND-TRIP A PREPROCESSING ARTIFACT

```
import pickle

artifact = {
    "vocab": {"<pad>": 0, "cat": 1, "dog": 2},
    "mean": 0.5,
    "std": 0.25,
}

blob = pickle.dumps(artifact)
print("serialised bytes:", len(blob))

restored = pickle.loads(blob)
print("vocab size:", len(restored["vocab"]))
print("dog id   :", restored["vocab"]["dog"])
```

OUTPUT

```
serialised bytes: 85
vocab size: 3
dog id   : 2
```

Pickling NumPy arrays to disk

NumPy arrays pickle cleanly, so a dict of weights survives a save/load round-trip exactly. In real code you would prefer `numpy.save/load (.npy)` for plain arrays, but pickle wins when you need to bundle arrays together with other Python objects.

SAVE A WEIGHT DICT, THEN RELOAD IT

```
import pickle, tempfile, os
import numpy as np

weights = {"w": np.arange(6).reshape(2, 3), "b": np.zeros(2)}

path = os.path.join(tempfile.mkdtemp(), "weights.pkl")
with open(path, "wb") as f:      # note the binary mode "wb"
    pickle.dump(weights, f)

with open(path, "rb") as f:
    loaded = pickle.load(f)

print("shapes:", {k: v.shape for k, v in loaded.items()})
print("equal :", np.array_equal(weights["w"], loaded["w"]))
os.remove(path)
```

OUTPUT

```
shapes: {'w': (2, 3), 'b': (2,)}
equal : True
```

The framework equivalent

Deep-learning frameworks wrap pickle in their own save/load helpers. The snippet below is *illustrative* (PyTorch is not installed here) but shows the identical mental model: one call to persist state, one to restore it.

ILLUSTRATIVE: TORCH.SAVE / TORCH.LOAD

```
import torch

# Save just the learned parameters (the recommended pattern).
torch.save(model.state_dict(), "model.pt")

# Later, rebuild the architecture, then load weights into it.
model = MyModel()
model.load_state_dict(torch.load("model.pt"))
model.eval()
```

Why it matters for AI - and a security warning: **never unpickle a file you did not create**. Unpickling can execute arbitrary code, so a malicious checkpoint downloaded from the internet is a remote-code-execution risk. This is exactly why safer formats like [safetensors](#) (weights only, no code) and [joblib](#) (efficient for large arrays) exist.

PART VII

Concurrency and Performance

CHAPTER 26

Threads, Processes and `concurrent.futures` for Data Loading

Data loading is often the real bottleneck in training: the GPU sits idle while the CPU decodes images or tokenises text. Python gives you two kinds of parallelism to hide that latency, and choosing between them comes down to one acronym: the **GIL**.

The GIL: threads vs processes

The Global Interpreter Lock lets only one thread run Python bytecode at a time. So **threads** win for I/O-bound work (downloading, reading files) because the GIL is released while waiting, but **processes** are needed for CPU-bound work (heavy pure-Python computation) since each process has its own interpreter and GIL.

- **Threads** - I/O-bound: downloading shards, reading files, calling APIs.
- **Processes** - CPU-bound: heavy pixel/audio decoding, pure-Python math.
- Most tensor math already runs in C and releases the GIL, so threads go further than beginners expect.

`ThreadPoolExecutor.map` for parallel I/O

`concurrent.futures` gives a single clean API for both pools. `executor.map` applies a function across an iterable concurrently and returns results **in input order** - ideal for preprocessing a batch of samples. We simulate I/O latency with `time.sleep`.

PARALLEL PREPROCESSING WITH A THREAD POOL

```

import time
from concurrent.futures import ThreadPoolExecutor

def load_sample(i):
    time.sleep(0.1)          # pretend this is a network / disk ←
    read
    return i * i

items = range(8)

start = time.perf_counter()
with ThreadPoolExecutor(max_workers=8) as pool:
    results = list(pool.map(load_sample, items))
elapsed = time.perf_counter() - start

print("results:", results)
print(f"8 x 0.1s work done in {elapsed:.2f}s (concurrent, not 0 ←
      .8s)")

```

OUTPUT

```

results: [0, 1, 4, 9, 16, 25, 36, 49]
8 x 0.1s work done in 0.10s (concurrent, not 0.8s)

```

submit + as_completed

When you want each result the moment it is ready (say, to update a progress bar) use `submit` to schedule work and `as_completed` to iterate over futures as they finish, in **completion order** rather than submission order.

PROCESS RESULTS AS THEY FINISH

```
import time, random
from concurrent.futures import ThreadPoolExecutor, as_completed

def fetch(url):
    time.sleep(random.uniform(0.02, 0.12))
    return f"{url} -> 200 OK"

urls = [f"shard-{i}" for i in range(5)]

with ThreadPoolExecutor(max_workers=5) as pool:
    futures = {pool.submit(fetch, u): u for u in urls}
    for fut in as_completed(futures):
        print("done:", fut.result())
```

OUTPUT

```
done: shard-0 -> 200 OK
done: shard-3 -> 200 OK
done: shard-4 -> 200 OK
done: shard-2 -> 200 OK
done: shard-1 -> 200 OK
```

Switching to processes for CPU work

The API is identical - just swap the executor class. The one catch is that process pools re-import your module, so the launch code must sit under an `if __name__ == '__main__':` guard. The snippet below is *illustrative* (process pools are awkward inside tiny scripts, but this is exactly the shape you use in a real training script).

ILLUSTRATIVE: PROCESSPOOLEXECUTOR FOR CPU-BOUND WORK

```

from concurrent.futures import ProcessPoolExecutor

def heavy_decode(chunk):
    # Pure-Python CPU work that the GIL would otherwise seriali
    se.
    return sum(i * i for i in range(chunk))

if __name__ == "__main__":          # required for process pools
    with ProcessPoolExecutor(max_workers=4) as pool:
        totals = list(pool.map(heavy_decode, [10_000] * 4))
        print(totals)

```

Why it matters for AI: this is exactly what a PyTorch [DataLoader\(num_workers=N\)](#) does under the hood - fan out sample loading across workers so the GPU never starves. Reach for threads on I/O-bound pipelines and processes only when the work is genuinely CPU-bound Python.

CHAPTER 27

async / await for Concurrent LLM and API Calls

When you call an LLM API, almost all the time is spent waiting for the network. **async/await** lets a single thread juggle hundreds of in-flight requests, making it the standard tool for batching prompts, building agents and serving inference.

Defining and running a coroutine

`async def` defines a *coroutine*; calling it returns an object that does nothing until awaited. `asyncio.run` starts the event loop and drives one coroutine to completion. `await` yields control while waiting, so other tasks can proceed.

A FIRST COROUTINE

```
import asyncio

async def call_model(prompt):
    await asyncio.sleep(0.1)           # stand-in for a network round-trip
    return f"response to: {prompt!r}"

async def main():
    reply = await call_model("hello")
    print(reply)

asyncio.run(main())
```

OUTPUT

```
response to: 'hello'
```

gather: fire many requests at once

`asyncio.gather` schedules many coroutines concurrently and waits for all of them, returning results in order. Ten requests that each take 0.1s finish in about 0.1s total, not 1.0s - the

whole point of `async`.

CONCURRENT REQUESTS WITH GATHER

```
import asyncio, time

async def call_model(prompt):
    await asyncio.sleep(0.1)
    return prompt.upper()

async def main():
    prompts = [f"q{i}" for i in range(10)]
    start = time.perf_counter()
    results = await asyncio.gather(*(call_model(p) for p in prompts))
    elapsed = time.perf_counter() - start
    print("results:", results)
    print(f"10 requests in {elapsed:.2f}s (concurrent, not 1.0s)")

asyncio.run(main())
```

OUTPUT

```
results: ['Q0', 'Q1', 'Q2', 'Q3', 'Q4', 'Q5', 'Q6', 'Q7', 'Q8', 'Q9']
10 requests in 0.10s (concurrent, not 1.0s)
```

Semaphore: respecting rate limits

Real APIs cap how many requests you may have in flight. An `asyncio.Semaphore` bounds concurrency: acquire it before each call and only `N` run at once, while the rest wait their turn. This is the idiomatic way to stay under a rate limit.

BOUNDING CONCURRENCY TO N AT A TIME

```

import asyncio, time

sem = asyncio.Semaphore(3)          # at most 3 concurrent calls
s

async def call_model(i):
    async with sem:
        await asyncio.sleep(0.1)
        return i

async def main():
    start = time.perf_counter()
    results = await asyncio.gather(*(call_model(i) for i in range(9)))
    elapsed = time.perf_counter() - start
    # 9 tasks, 3 at a time, 0.1s each => ~3 waves => ~0.3s
    print("results:", results)
    print(f"9 calls, limit 3, took {elapsed:.2f}s (~3 waves)")

asyncio.run(main())

```

OUTPUT

```

results: [0, 1, 2, 3, 4, 5, 6, 7, 8]
9 calls, limit 3, took 0.30s (~3 waves)

```

What a real async LLM client looks like

The pattern is identical with a real SDK: build an async client, wrap each call in a semaphore, and [gather](#) them. The snippet below is *illustrative* only - no network call is made - but it mirrors the shape of production batch-inference code.

ILLUSTRATIVE: CONCURRENT REAL API CALLS


```

import asyncio
from some_llm_sdk import AsyncClient      # e.g. an async Anthr ←
    opic/OpenAI client

client = AsyncClient()
sem = asyncio.Semaphore(5)                # honour the provider' ←
    s rate limit

async def ask(prompt):
    async with sem:
        resp = await client.messages.create(
            model="claude-opus-4-8",
            max_tokens=256,
            messages=[{"role": "user", "content": prompt}],
        )
        return resp.content[0].text

async def main(prompts):
    return await asyncio.gather(*(ask(p) for p in prompts))

answers = asyncio.run(main(["Summarise X", "Translate Y", "Clas ←
    sify Z"]))

```

Why it matters for AI: batch inference, agent loops and retrieval pipelines all issue many network calls whose cost is latency, not CPU. [asyncio.gather](#) for concurrency plus a [Semaphore](#) for rate limiting is the backbone of fast, well-behaved LLM clients.

PART VIII

The Numeric AI Stack

CHAPTER 28

NumPy Arrays, dtypes and Vectorization

NumPy is the foundation of the entire Python AI stack. A tensor in PyTorch, TensorFlow or JAX is essentially a NumPy array with a GPU and autograd bolted on, so the mental model you build here transfers directly. The core object is the **ndarray**: a contiguous block of numbers with a single **dtype** and a **shape**.

Creating arrays

You rarely type numbers by hand. **zeros**, **ones**, **arange** and **linspace** build the weight matrices, index ranges and learning-rate schedules you actually use.

WAYS TO MAKE AN ARRAY

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])
print("from list:\n", a)
print("zeros:", np.zeros(3))
print("ones (2x2):\n", np.ones((2, 2)))
print("arange:", np.arange(0, 10, 2))
print("linspace:", np.linspace(0.0, 1.0, 5))
```

OUTPUT

```
from list:
[[1 2 3]
 [4 5 6]]
zeros: [0. 0. 0.]
ones (2x2):
[[1. 1.]
 [1. 1.]]
arange: [0 2 4 6 8]
linspace: [0.    0.25 0.5   0.75 1.   ]
```

dtype: why models use float32

Every array has one `dtype`. Deep-learning models default to `float32` (not Python's 64-bit float) because it halves memory and doubles throughput on GPUs with negligible accuracy loss.

Knowing when a cast happens saves you from silent slowdowns.

DYPES AND CASTING

```
import numpy as np

x = np.array([1.0, 2.0, 3.0])
print("default dtype:", x.dtype)           # float64
w = np.ones(3, dtype=np.float32)
print("model weights:", w.dtype)
print("bytes each:", x.nbytes, "vs", w.nbytes)
ids = np.array([5, 1, 9], dtype=np.int64) # token ids are ints
print("ids dtype:", ids.dtype)
```

OUTPUT

```
default dtype: float64
model weights: float32
bytes each: 24 vs 12
ids dtype: int64
```

shape, ndim, size and reshape

The `shape` is the grammar of tensor code. A batch of images might be `(32, 3, 224, 224)`; a batch of token embeddings (`batch, seq_len, dim`). `reshape` rearranges the same data without copying it, and `-1` means "infer this axis".

INSPECTING AND RESHAPING

```
import numpy as np

a = np.arange(12)
print("shape:", a.shape, "ndim:", a.ndim, "size:", a.size)
b = a.reshape(3, 4)
print("reshaped:\n", b)
print("flatten with -1:", b.reshape(-1).shape)
print("add batch axis:", a.reshape(1, -1).shape)
```

OUTPUT

```

shape: (12,) ndim: 1 size: 12
reshaped:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
flatten with -1: (12,)
add batch axis: (1, 12)

```

Vectorization beats Python loops

The golden rule of numeric Python: *never loop over elements if an array operation exists*. Vectorized operations run in optimized C, so they are typically 10-100x faster and read like the math they implement.

LOOP VS VECTORIZED (SAME RESULT, ONE IS FAST)

```

import numpy as np, time

x = np.arange(1_000_000, dtype=np.float64)

t0 = time.perf_counter()
loop = [v * 2.0 + 1.0 for v in x]      # slow, pure Python
t1 = time.perf_counter()
vec = x * 2.0 + 1.0                    # fast, vectorized C
t2 = time.perf_counter()

print("same result:", np.allclose(loop, vec))
print(f"loop: {t1 - t0:.4f}s")
print(f"vector:{t2 - t1:.4f}s")

```

OUTPUT

```

same result: True
loop: 0.1227s
vector:0.0028s

```

Aggregations along an axis

Reductions like `sum`, `mean` and `max` take an `axis` argument. `axis=0` collapses rows (per-feature stats), `axis=1` collapses columns (per-sample stats). This is how you compute batch statistics

and pool over sequence dimensions.

REDUCTIONS WITH AXIS

```
import numpy as np

batch = np.array([[1.0, 2.0, 3.0],
                  [4.0, 5.0, 6.0]])  # (2 samples, 3 features)
print("grand mean:", batch.mean())
print("per-feature mean (axis=0):", batch.mean(axis=0))
print("per-sample sum (axis=1):", batch.sum(axis=1))
print("argmax per sample:", batch.argmax(axis=1))
```

OUTPUT

```
grand mean: 3.5
per-feature mean (axis=0): [2.5 3.5 4.5]
per-sample sum (axis=1): [ 6. 15.]
argmax per sample: [2 2]
```

Why it matters for AI: every forward pass is array creation, reshaping and vectorized math over a fixed dtype. If you can read shapes and reach for an axis-aware reduction instead of a for-loop, you can read almost any model's code.

CHAPTER 29

Broadcasting, Indexing and Linear Algebra: the Math of a Layer

A neural network layer is mostly one line of linear algebra: $y = X @ W + b$. To write and debug it you need three skills: **broadcasting** (combining arrays of different shapes), **indexing** (pulling out the elements you want), and the **matmul** operator `@`.

Broadcasting: adding a bias to a batch

Broadcasting lets NumPy stretch a smaller array across a larger one without copying. Adding a bias vector of shape `(features,)` to a batch of shape `(batch, features)` just works, because the trailing dimensions match.

BROADCASTING A BIAS ACROSS A BATCH

```
import numpy as np

batch = np.zeros((3, 4))          # 3 samples, 4 features
bias = np.array([10, 20, 30, 40]) # one bias per feature
print("batch + bias:\n", batch + bias)
# column vector broadcasts the other way:
scale = np.array([[1], [2], [3]]) # shape (3, 1) -> per-sample
print("per-sample scale:\n", (batch + bias) * scale)
```

OUTPUT

```
batch + bias:
[[10. 20. 30. 40.]
 [10. 20. 30. 40.]
 [10. 20. 30. 40.]]
per-sample scale:
[[ 10.  20.  30.  40.]
 [ 20.  40.  60.  80.]
 [ 30.  60.  90. 120.]]
```

Slicing and boolean masking

Slices select contiguous regions; boolean masks select elements by condition. Masking is how you apply a ReLU, filter padded tokens, or clip outliers, all without a loop.

SLICES AND MASKS

```
import numpy as np

a = np.arange(1, 13).reshape(3, 4)
print("first row:", a[0])
print("last column:", a[:, -1])
print("top-left 2x2:\n", a[:2, :2])

mask = a > 6
print("mask:\n", mask)
relu = np.where(a - 6 > 0, a - 6, 0)    # a mini ReLU
print("relu(a-6):\n", relu)
```

OUTPUT

```
first row: [1 2 3 4]
last column: [ 4  8 12]
top-left 2x2:
[[1 2]
 [5 6]]
mask:
[[False False False False]
 [False False  True  True]
 [ True  True  True  True]]
relu(a-6):
[[0 0 0 0]
 [0 0 1 2]
 [3 4 5 6]]
```

Fancy indexing: gathering rows

Indexing with an array of integers gathers arbitrary rows in any order, with repeats. This is exactly an **embedding lookup**: token ids index into an embedding table.

EMBEDDING LOOKUP BY FANCY INDEXING


```
import numpy as np

embed = np.round(np.arange(15).reshape(5, 3) / 10, 1) # vocab=5, dim=3
print("embedding table:\n", embed)
token_ids = np.array([0, 2, 2, 4])
print("looked up:\n", embed[token_ids]) # shape (4, 3)
```

OUTPUT

```
embedding table:
[[0.  0.1 0.2]
 [0.3 0.4 0.5]
 [0.6 0.7 0.8]
 [0.9 1.  1.1]
 [1.2 1.3 1.4]]
looked up:
[[0.  0.1 0.2]
 [0.6 0.7 0.8]
 [0.6 0.7 0.8]
 [1.2 1.3 1.4]]
```

The matmul operator @ and a linear layer

@ is matrix multiplication. Combined with broadcasting for the bias, it is a complete dense layer. Watch the shapes: (batch, in) @ (in, out) -> (batch, out).

A BATCHED LINEAR LAYER $Y = X @ W + B$

```
import numpy as np
rng = np.random.default_rng(0)

X = rng.normal(size=(2, 3)) # batch=2, in_features=3
W = rng.normal(size=(3, 4)) # in=3, out=4
b = np.zeros(4) # one bias per output

y = X @ W + b
print("X:", X.shape, " W:", W.shape, " -> y:", y.shape)
print("y:\n", np.round(y, 3))
```

OUTPUT

```
X: (2, 3)  W: (3, 4)  ->  y: (2, 4)
y:
[[-0.552 -0.355 -0.13  -0.333]
 [ 0.02  -0.188  0.975 -0.13  ]]
```

A little linear algebra

`np.linalg` gives you norms (for gradient clipping and regularization), dot products (for cosine similarity), and solvers. Cosine similarity between embeddings is one of the most common operations in retrieval and RAG systems.

COSINE SIMILARITY BETWEEN TWO VECTORS

```
import numpy as np

u = np.array([1.0, 2.0, 2.0])
v = np.array([2.0, 0.0, 1.0])
cos = (u @ v) / (np.linalg.norm(u) * np.linalg.norm(v))
print("dot:", u @ v)
print("|u|:", round(float(np.linalg.norm(u)), 3))
print("cosine similarity:", round(float(cos), 3))
```

OUTPUT

```
dot: 4.0
|u|: 3.0
cosine similarity: 0.596
```

Why it matters for AI: forward passes, attention scores and similarity search are all matmuls plus broadcast adds. Fancy indexing *is* the embedding layer. Get comfortable reading shapes through an `@` and you can trace data through any model.

CHAPTER 30

Random, Statistics and Activation Functions from Scratch

Randomness (for weight initialization and sampling) and statistics (for normalization) sit at the heart of training. In this chapter we build the classic activation and loss functions in pure NumPy so you know exactly what a framework computes under the hood.

Reproducible randomness

Always seed your generator. The modern API is `np.random.default_rng(seed)`, which gives an isolated stream, so your experiments are reproducible. Weight initialization typically draws from a scaled normal distribution.

SEEDED INIT AND SAMPLING

```
import numpy as np
rng = np.random.default_rng(0)

W = rng.normal(0, 0.02, size=(2, 3)) # GPT-style small init
print("weights:\n", np.round(W, 3))
print("uniform:", np.round(rng.uniform(size=3), 3))
print("random choice:", rng.choice([10, 20, 30], size=4))
```

OUTPUT

```
weights:
[[ 0.003 -0.003  0.013]
 [ 0.002 -0.011  0.007]]
uniform: [0.607 0.729 0.544]
random choice: [20 30 10 30]
```

Statistics and normalization

Normalizing inputs (zero mean, unit variance) makes training

stable. The z-score below is the same computation LayerNorm and BatchNorm perform, minus the learnable scale and shift.

Z-SCORE NORMALIZATION (A MINI LAYERNORM)

```
import numpy as np
rng = np.random.default_rng(1)

x = rng.normal(5, 3, size=(2, 4))      # not centered
mu = x.mean(axis=1, keepdims=True)
sd = x.std(axis=1, keepdims=True)
norm = (x - mu) / (sd + 1e-5)
print("mean before:", np.round(x.mean(axis=1), 2))
print("mean after :", np.round(norm.mean(axis=1), 6))
print("std  after :", np.round(norm.std(axis=1), 3))
```

OUTPUT

```
mean before: [5.15 6.05]
mean after  : [-0.  0.]
std  after  : [1. 1.]
```

Sigmoid and ReLU

Activations are just elementwise functions over an array. Vectorization means the whole tensor is transformed at once.

TWO ACTIVATION FUNCTIONS

```
import numpy as np

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def relu(z):
    return np.maximum(0.0, z)

z = np.array([-2.0, -0.5, 0.0, 0.5, 2.0])
print("sigmoid:", np.round(sigmoid(z), 3))
print("relu   :", relu(z))
```

OUTPUT

```
sigmoid: [0.119 0.378 0.5   0.622 0.881]
relu   : [0.  0.  0.  0.5 2. ]
```

Numerically stable softmax

Softmax turns logits into probabilities. The naive version overflows on large logits, so the standard trick is to subtract the max first, this changes nothing mathematically but keeps `exp` in range. Every framework does exactly this.

SOFTMAX WITH THE MAX-SUBTRACTION TRICK

```
import numpy as np

def softmax(logits, axis=-1):
    z = logits - logits.max(axis=axis, keepdims=True) # stability
    e = np.exp(z)
    return e / e.sum(axis=axis, keepdims=True)

logits = np.array([[2.0, 1.0, 0.1],
                   [0.0, 0.0, 5.0]])
probs = softmax(logits)
print("probs:\n", np.round(probs, 3))
print("rows sum to 1:", np.round(probs.sum(axis=1), 6))
```

OUTPUT

```
probs:
[[0.659 0.242 0.099]
 [0.007 0.007 0.987]]
rows sum to 1: [1. 1.]
```

Cross-entropy loss

Cross-entropy measures how far predicted probabilities are from the true labels. It combines fancy indexing (to pull the probability of the correct class) with a log and a mean, the loss that trains most classifiers and language models.

CROSS-ENTROPY FROM PROBABILITIES

```

import numpy as np

probs = np.array([[0.7, 0.2, 0.1],
                  [0.1, 0.1, 0.8]])
labels = np.array([0, 2])          # correct class per sample
le = 1
n = len(labels)
correct_p = probs[np.arange(n), labels]    # fancy indexing
loss = -np.mean(np.log(correct_p + 1e-9))
print("prob of correct class:", correct_p)
print("cross-entropy loss:", round(float(loss), 4))

```

OUTPUT

```

prob of correct class: [0.7 0.8]
cross-entropy loss: 0.2899

```

Why it matters for AI: softmax, ReLU and cross-entropy are the building blocks of nearly every model. Written in NumPy they are a few lines each, the framework versions add autograd and GPU support but compute the same thing, stability tricks included.

CHAPTER 31

pandas Series and DataFrame: Loading Datasets

Before data becomes tensors it usually lives in a table: CSVs, logs, feature stores. **pandas** is the standard tool for loading, inspecting and slicing that tabular data. Its two types are the 1-D [Series](#) and the 2-D [DataFrame](#) (a dict of aligned Series sharing an index).

Series: a labeled array

A SERIES IS VALUES PLUS AN INDEX

```
import pandas as pd

s = pd.Series([0.9, 0.4, 0.7], index=["cat", "dog", "fox"])
print(s)
print("by label:", s["dog"])
print("mean:", round(s.mean(), 3))
print("above 0.5:\n", s[s > 0.5])
```

OUTPUT

```
cat    0.9
dog    0.4
fox    0.7
dtype: float64
by label: 0.4
mean: 0.667
above 0.5:
  cat    0.9
  fox    0.7
dtype: float64
```

Building a DataFrame

You will usually read a DataFrame with `pd.read_csv(...)`, but constructing one from a dict is the clearest way to learn it. Think of each key as a column of a dataset.

A SMALL DATASET AS A DATAFRAME

```
import pandas as pd

df = pd.DataFrame({
    "text_len": [12, 45, 7, 33, 20],
    "label":    [0, 1, 0, 1, 0],
    "source":   ["web", "book", "web", "book", "web"],
})
print(df)
print("\ndtypes:\n", df.dtypes)
print("\nshape:", df.shape)
```

OUTPUT

```
   text_len  label source
0        12     0    web
1        45     1   book
2         7     0    web
3        33     1   book
4        20     0    web
```

```
dtypes:
text_len    int64
label       int64
source      str
dtype: object
```

```
shape: (5, 3)
```

Inspecting: head and describe

The first thing you do with any dataset is look at it. [head](#) shows the top rows; [describe](#) gives summary statistics for the numeric columns, an instant sanity check on ranges and scale.

QUICK LOOK AT THE DATA


```
import pandas as pd

df = pd.DataFrame({
    "text_len": [12, 45, 7, 33, 20],
    "label":    [0, 1, 0, 1, 0],
})
print(df.head(2))
print("\nsummary:\n", df.describe().round(2))
```

OUTPUT

```
   text_len  label
0        12      0
1        45      1

summary:
      text_len  label
count      5.00  5.00
mean      23.40  0.40
std       15.57  0.55
min        7.00  0.00
25%       12.00  0.00
50%       20.00  0.00
75%       33.00  1.00
max       45.00  1.00
```

Selecting with loc and iloc

`loc` selects by label, `iloc` selects by integer position. Pick columns with a list, and rows with a slice or a mask. This is how you separate features from labels before training.

ROWS, COLUMNS, FEATURES VS LABELS

```
import pandas as pd

df = pd.DataFrame({
    "text_len": [12, 45, 7, 33, 20],
    "n_words":  [2, 8, 1, 6, 4],
    "label":    [0, 1, 0, 1, 0],
})
X = df[["text_len", "n_words"]]      # feature columns
y = df["label"]                     # target column
print("features:\n", X.iloc[:2])
print("\nfirst label by iloc:", y.iloc[0])
print("label via loc:", df.loc[3, "label"])
```

OUTPUT

```
features:
   text_len  n_words
0         12         2
1         45         8

first label by iloc: 0
label via loc: 1
```

Filtering rows by condition

Boolean masks work on DataFrames just like on NumPy arrays, and you can combine conditions with `&` and `|` (each condition in parentheses). This is your everyday tool for subsetting data.

BOOLEAN FILTERING

```
import pandas as pd

df = pd.DataFrame({
    "text_len": [12, 45, 7, 33, 20],
    "label":    [0, 1, 0, 1, 0],
})
long_pos = df[(df["text_len"] > 15) & (df["label"] == 1)]
print(long_pos)
print("\ncount per label:\n", df["label"].value_counts())
```

OUTPUT

```
   text_len  label
1         45      1
3         33      1

count per label:
label
0      3
1      2
Name: count, dtype: int64
```

Why it matters for AI: real projects spend most of their time in data wrangling, not modeling. [head/describe/loc/masks](#) are how you understand and slice a dataset before a single tensor is created, and `df[features]`, `df[target]` is the split every training script begins with.

CHAPTER 32

pandas Cleaning, Grouping and Feature Engineering

Raw data is messy: missing values, wrong scales, categorical strings a model cannot consume. This chapter covers the transforms that turn a table into model-ready features.

Missing values

`isna` finds gaps, `fillna` imputes them (often with the column mean), and `dropna` removes them. Leaving NaNs in will poison your gradients, so this is step one of cleaning.

DETECT AND FILL MISSING VALUES

```
import pandas as pd
import numpy as np

df = pd.DataFrame({"age": [25, np.nan, 40, np.nan, 30]})
print("missing per column:\n", df.isna().sum())
df["age"] = df["age"].fillna(df["age"].mean())
print("\nafter fillna:\n", df.round(1))
```

OUTPUT

```
missing per column:
age      2
dtype: int64

after fillna:
   age
0  25.0
1  31.7
2  40.0
3  31.7
4  30.0
```

New columns with vectorized ops and apply

Feature engineering is creating new columns from existing

ones. Prefer vectorized expressions; use `map/apply` for logic that does not vectorize cleanly. Both return a new Series you assign back to the frame.

DERIVED FEATURES

```
import pandas as pd

df = pd.DataFrame({"chars": [40, 10, 200, 75]})
df["words"] = (df["chars"] / 5).round().astype(int)  # vectorized
df["bucket"] = df["chars"].apply(
    lambda c: "long" if c > 100 else "short"          # apply
)
print(df)
```

OUTPUT

	chars	words	bucket
0	40	8	short
1	10	2	short
2	200	40	long
3	75	15	short

Grouping and aggregation

`groupby` splits rows into groups and aggregates each, the table equivalent of a pivot report. It answers questions like "what is the mean label per data source?", essential for spotting bias and class imbalance.

GROUPBY + AGG

```
import pandas as pd

df = pd.DataFrame({
    "source": ["web", "book", "web", "book", "web"],
    "len":     [12, 45, 7, 33, 20],
    "label":   [0, 1, 0, 1, 0],
})
summary = df.groupby("source").agg(
    n=("label", "size"),
    mean_len=("len", "mean"),
    pos_rate=("label", "mean"),
)
print(summary.round(2))
```

OUTPUT

	n	mean_len	pos_rate
source			
book	2	39.0	1.0
web	3	13.0	0.0

One-hot encoding categoricals

Models need numbers, not strings. `pd.get_dummies` expands a categorical column into 0/1 indicator columns, one per category, the standard encoding for non-ordinal features.

GET_DUMMIES FOR CATEGORICAL FEATURES

```
import pandas as pd

df = pd.DataFrame({"color": ["red", "blue", "red", "green"]})
onehot = pd.get_dummies(df, columns=["color"], dtype=int)
print(onehot)
```

OUTPUT

	color_blue	color_green	color_red
0	0	0	1
1	1	0	0
2	0	0	1
3	0	1	0

Scaling a numeric column

Features on wildly different scales make optimization hard. Standardizing each numeric column to zero mean and unit variance, exactly as in the NumPy chapter, is a routine final step before handing data to a model.

STANDARDIZE A FEATURE COLUMN

```
import pandas as pd

df = pd.DataFrame({"income": [30000, 80000, 55000, 120000]})
col = df["income"]
df["income_z"] = (col - col.mean()) / col.std()
print(df.round(3))
```

OUTPUT

	income	income_z
0	30000	-1.075
1	80000	0.228
2	55000	-0.423
3	120000	1.270

Why it matters for AI: fillna, get_dummies and standardization are the classic tabular preprocessing pipeline, and scikit-learn transformers do exactly these steps. groupby is your fastest way to audit a dataset for imbalance and leakage before training.

CHAPTER 33

Visualization and the Deep-Learning Frameworks

This closing chapter connects the Python features from the whole book to the frameworks you will actually use. The punchline: PyTorch and friends invent very little new *language*, they lean on dunder methods, decorators, context managers and operator overloading you have already seen.

Plotting a loss curve (matplotlib, illustrative)

matplotlib is the default way to visualize training. The snippet below is illustrative (matplotlib is not installed in this build), but this is the exact idiom for saving a loss curve to a file during training.

ILLUSTRATIVE: SAVE A TRAINING LOSS CURVE

```
import matplotlib.pyplot as plt    # illustrative, not executed
    here

steps = list(range(0, 100, 10))
loss = [2.5, 1.8, 1.3, 1.0, 0.8, 0.7, 0.6, 0.55, 0.52, 0.5]

plt.figure(figsize=(5, 3))
plt.plot(steps, loss, marker="o")
plt.xlabel("step"); plt.ylabel("loss"); plt.title("Training loss")
plt.tight_layout()
plt.savefig("loss.png", dpi=150)    # write to disk, view later
```

The nn.Module pattern is just `__init__` + `__call__`

A PyTorch layer is a class: `__init__` registers parameters, `forward` defines the math, and calling the instance `layer(x)` works because `nn.Module` implements `__call__` (which then invokes `f`

orward)). This is the *exact* dunder pattern from the objects chapter, illustrative below since torch is not installed.

ILLUSTRATIVE: A REAL PYTORCH MODULE

```
import torch                                # illustrative, not executed ↵
    here
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, d_in, d_hidden, d_out):
        super().__init__()
        self.fc1 = nn.Linear(d_in, d_hidden)
        self.fc2 = nn.Linear(d_hidden, d_out)

    def forward(self, x):
        x = torch.relu(self.fc1(x))         # same relu as our NumPy ↵
        one
        return self.fc2(x)

model = MLP(3, 8, 2)
y = model(torch.randn(4, 3))               # __call__ -> forward
print(y.shape)                             # torch.Size([4, 2])
```

Decorators and context managers in real training

Turning off gradient tracking during evaluation uses the same two features from earlier chapters: `@torch.no_grad()` as a **decorator** on a function, or `with torch.no_grad():` as a **context manager**. Same tool, two syntaxes.

ILLUSTRATIVE: NO_GRAD AS DECORATOR AND CONTEXT

```

import torch                                # illustrative, not executed ↵
    here

@torch.no_grad()                            # decorator form
def evaluate(model, x):
    return model(x).argmax(dim=-1)

# ... inside a training loop ...
for step in range(100):
    with torch.no_grad():                    # context-manager form
        val_preds = model(val_x)            # no autograd graph built ↵
        t
    # train step here would use gradients as normal

```

A verified gradient-descent step in NumPy

To keep this chapter grounded, here is a real (executed) training step for linear regression in pure NumPy: forward pass, mean-squared-error loss, analytic gradient, and a weight update. Watch the loss fall, this is what autograd automates.

ONE EPOCH LOOP OF GRADIENT DESCENT

```

import numpy as np
rng = np.random.default_rng(0)

X = rng.normal(size=(50, 3))
true_w = np.array([2.0, -1.0, 0.5])
y = X @ true_w + 0.1 * rng.normal(size=50) # noisy targets

w = np.zeros(3)
lr = 0.1
for step in range(0, 101):
    pred = X @ w                                # forward
    err = pred - y
    loss = np.mean(err ** 2)                    # MSE
    grad = 2.0 / len(y) * (X.T @ err)          # gradient
    w -= lr * grad                              # update
    if step % 25 == 0:
        print(f"step {step:3d}  loss={loss:.4f}")
print("learned w:", np.round(w, 2))

```

OUTPUT

```
step    0  loss=6.0123
step   25  loss=0.0094
step   50  loss=0.0087
step   75  loss=0.0087
step  100  loss=0.0087
learned w: [ 2.02 -1.02  0.5 ]
```

The whole book, in one sentence

- **Containers & comprehensions** build datasets and batches.
- **Generators** stream data too big for memory.
- **Dunder methods** (`__call__`) make models callable like functions.
- **Decorators & context managers** toggle behavior like `no_grad`.
- **Dataclasses & type hints** describe configs and tensors.
- **NumPy** vectorization is the math every framework accelerates.

Why it matters for AI: once you see that `model(x)` is `__call__`, that `@torch.no_grad()` is a decorator, and that a tensor is an ndarray with autograd, framework code stops looking like magic and starts looking like the Python you already know.